electronics

MDPI

Article

Multi-Head Attention DQN and Dynamic Priority for Path Planning of Unmanned Aerial Vehicles Oriented to Penetration

Liuyu Cheng and Wei Shang *

School of Mechanical Engineering, Hubei University of Technology, Wuhan 430000, China; 2310110310@hbut.edu.cn
* Correspondence: 20191018@mail.hbut.edu.cn

Abstract

Unmanned aerial vehicle (UAV) penetration missions in hostile environments face significant challenges due to dense threat coverage, dynamic defense systems, and the need for real-time decision-making under uncertainty. Traditional path planning methods suffer from computational intractability in high-dimensional spaces, while existing deep reinforcement learning approaches lack efficient feature extraction and sample utilization mechanisms for threat-dense scenarios. To address these limitations, this paper presents an enhanced Deep Q-Network (DQN) framework integrating multi-head attention mechanisms with dynamic priority experience replay for autonomous UAV path planning. The proposed architecture employs four specialized attention heads operating in parallel to extract proximity, danger, alignment, and threat density features, enabling selective focus on critical environmental aspects. A dynamic priority mechanism adaptively adjusts sampling strategies during training, prioritizing informative experiences in early exploration while maintaining balanced learning in later stages. Experimental results demonstrate that the proposed method achieves 94.3% mission success rate in complex penetration scenarios, representing 7.1–17.5% improvement over state-of-the-art baselines with 2.2× faster convergence. The approach shows superior robustness in high-threat environments and meets real-time operational requirements with 18.3 ms inference latency, demonstrating its practical viability for autonomous UAV penetration missions.

Keywords: unmanned aerial vehicle; path planning; multi-head attention; deep reinforcement learning; prioritized experience replay

Academic Editor: Mahmut Reyhanoglu

Received: 4 December 2025
Revised: 20 December 2025
Accepted: 27 December 2025
Published: 29 December 2025

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

1.1. Research Background and Motivation

1.1.1. Application Requirements of UAV Penetration Missions

Unmanned aerial vehicles are essential components in current and future military operations, especially those involving penetration that entail flying through deeply defended hostile zones. In such operations, UAVs must autonomously plan safe routes while avoiding radar detection and air defense threats. The increasing sophistication of modern defense systems with overlapping threat coverage calls for intelligent route planning in dynamically challenging environments [1,2]. Too much data is involved in real-

time operations to be processed manually by the pilot in either current UAV technology or traditional manually operated platforms.

1.1.2. Challenges in Path Planning

Penetration-related UAV missions entail path planning tasks that are more complex compared to other navigation tasks. The first challenge in solving these missions lies in optimizing various competing tasks concurrently. The tasks include finding the shortest possible distance to fly, staying out of threat zones, optimizing kinematics, and maintaining the solution within real-time computational requirements. The computational requirement becomes exponentially complex with the increase in threat instances [3,4].

1.1.3. Limitations of Existing Methods

The existing path planning techniques are found to be highly inefficient in penetration missions. The graph-based techniques, A* and Dijkstra's algorithm, are guaranteed to be optimal in static environments but are inefficient due to exponential scaling with the expansion of state space and lack of dynamics to counter threats. Sampling-based techniques such as RRT and RRT* are probabilistically complete; however, these techniques produce suboptimal solutions with thin corridors between threat regions. Recent works in deep reinforcement learning are promising, and DQN learning has shown autonomous decision-making in complex settings [5,6]. In standard DQN learning models, the environment's features are processed with uniform relevance without prioritizing threats. In addition to this, the standard experience replay process ignores experiences that are highly valuable during learning; these experiences are significant to efficiently master penetration missions.

1.2. Related Work

1.2.1. Traditional Path Planning Methods

Classical planning algorithms set the stage with established approaches to autonomous navigation. Discrete graph search algorithms build graphs that model the environment and use rigorous search approaches to find paths that avoid collisions. Debnath et al. combined geometric obstacle avoidance with genetic algorithm solving of the traveling salesman problem to optimize UAV routes and proved that such hybrid models are efficient in structured environments. They are theoretically complete models that are efficient in lower configuration space dimensions but inefficient in higher environments with higher threats as those posed to penetration missions.

1.2.2. Deep Reinforcement Learning in Path Planning

The application of deep reinforcement learning techniques to the problem of path planning has received extensive attention due to its potential to obtain near-optimal solutions to the problem through interaction with the environment. Value-function-based approaches, such as deep Q-networks, approximate action-value functions using deep neural networks to make sequential decisions for autonomous agents in complex state spaces. Wang et al. introduced an improved double deep Q-network algorithm to solve the problem of mobile robot path planning, which incorporated dueling architectures to separate state value and action advantage estimation, and employed prioritized replay buffers to enhance sample efficiency, achieving 15% improvement in navigation success rate. Ali et al. presented an extensive review article covering various approaches to UAV path planning with increased emphasis on the application of reinforcement learning techniques to cope with dynamic and uncertain environments. Wang et al. designed a deep reinforcement learning-based approach for autonomous navigation in unknown environments,

utilizing convolutional neural networks to process raw sensor inputs and demonstrating effective collision avoidance across indoor and outdoor scenarios. Policy gradient approaches such as PPO and A3C enable direct policy optimization, with applications extended to energy-efficient UAV path planning [7]. Yu et al. presented works that use deep reinforcement learning to enable dynamic scene path planning with increased emphasis on adapting to time-varying environmental factors [8]. Xu et al. presented works on multi-UAV path planning in communication-denied environments with potential scaling capacities [9]. More recently, Liu et al. investigated AoI-minimal clustering, transmission and trajectory co-design for UAV-assisted wireless powered communication networks, providing insights into joint optimization of UAV trajectories [10]. These state-of-the-art techniques still lack mature designs to enable predictive attention to peer-critical features in threat-dense penetration missions.

1.2.3. Attention Mechanisms in Decision-Making

Attention mechanisms have proven to be useful technologies that improve the architecture designs proposed by neural networks. The use of attention mechanisms within reinforcement learning environments enables an agent to focus on the most important observations from the environment with higher dimensionality. Hu et al. presented an extensive analysis of how attention mechanisms can be useful in multi-agent reinforcement learning environments to facilitate coordination among entities within these environments [11]. Multi-head attention architecture divides the representation space into various subspaces to enable the computation of diverse attention mechanisms in such environments. The use of multi-level attention mechanisms within deep reinforcement learning recommendation environments was shown to be effective by Wang et al. [12]. The application of attention mechanisms within UAV swarm task allocation environments was presented by Qian et al. [13]. The combination of multi-head attention mechanisms with value-based reinforcement learning has yet to be fully investigated within penetration-oriented path planning.

The prioritized experience replay learning mechanism, presented by Schaul et al., brought about a significant change in the process of training reinforcement learning models by selecting experiences in proportion to the error difference [14]. Pi et al. extended prioritized DQN to optimize network problems to improve sample efficiency and results [15]. Dong et al. analyzed the current state-of-the-art deep reinforcement learning techniques for unmanned aerial vehicle path planning. The adjustment strategies to modify the priority distributions during training are still unexplored.

1.3. Research Objectives and Contributions

1.3.1. Main Innovations

The key novelty of this work lies in the synergistic integration of multi-head attention mechanisms with dynamic priority experience replay within the DQN framework for UAV penetration path planning. Unlike existing methods that process threat information uniformly, our approach employs four specialized attention heads to simultaneously extract proximity, lethality, alignment, and density features, while the dynamic priority mechanism adaptively adjusts sampling strategies to accelerate learning from sparse successful experiences. The novel element here is the application of multi-head attention techniques within the architecture of the value network to allow the agent to consider various aspects of threats concurrently by means of multiple attention heads. The proposed architecture adopts the use of dynamic priority to determine sampling approaches during training to prioritize experiences during initial exploration stages while shifting to balanced learning afterwards.

1.3.2. Expected Contributions

This research offers the following contributions to the research area. First, it pushes forward the theoretical understanding about how multi-head attention mechanisms improve the estimation of the value function in environments with many threats. Second, it proposes an adaptive adjustment strategy that improves the efficiency and robustness over static prioritization approaches. Third, it offers an operational solution that has been tested in simulations with various threat settings to be useful in real-world penetration operations.

1.3.3. Paper Organization

The rest of this paper has been organized as follows. Section 2 explains the problem formulation, methodology, and various training processes. Section 3 reports results with various comparative tests and more complex scenarios. Section 4 provides implications and limitations. Section 5 provides conclusions and suggestions on further research.

2. Materials and Methods

2.1. Simulation Environment and Scenario Configuration

2.1.1. Threat Environment Modeling

The threat environment encompasses two fundamental types of hazards: radar detection systems and anti-aircraft fire zones. The radar detection system has a probabilistic model whereby the probability of detection is exponentially lower with the increase in distance from the radar origin and depends on the radar cross-section area of the UAV. Each radar station has parameters such as Rd (maximum detection range), detection angle denoted by θ_d, and baseline detection probability P0. The detection probability formula depending on the UAV distance to the station is $P(r) = P_0 e^{-\alpha r / R_d}$, where r denotes the distance from the UAV to the radar station and α is the attenuation coefficient. The formula applies if the UAV lies within the Rd detection range. Anti-aircraft fire zones take into account the areas surrounding threats with lethal radius R1 and damage gradients up to the effective radius Re. The threat intensities are spread according to an inverse square law formula, whereby the damage decreases with respect to the square of the distance from the origin. The need to apply adaptive control techniques to allow UAVs to move within complex threat environments was highlighted by Yin et al. [16]. As shown in Figure 1 below, the simulation environment models the penetration mission with threats spread over spatial zones that form coverage areas challenging the UAV to exploit safe passage corridors. Figure 1 illustrates a representative scenario with radar detection zones (blue) and anti-aircraft fire zones (orange), demonstrating the overlapping threat coverage and the resulting narrow safe corridors that the UAV trajectory (green) must navigate. This visualization clarifies the spatial complexity that motivates our attention-based approach.

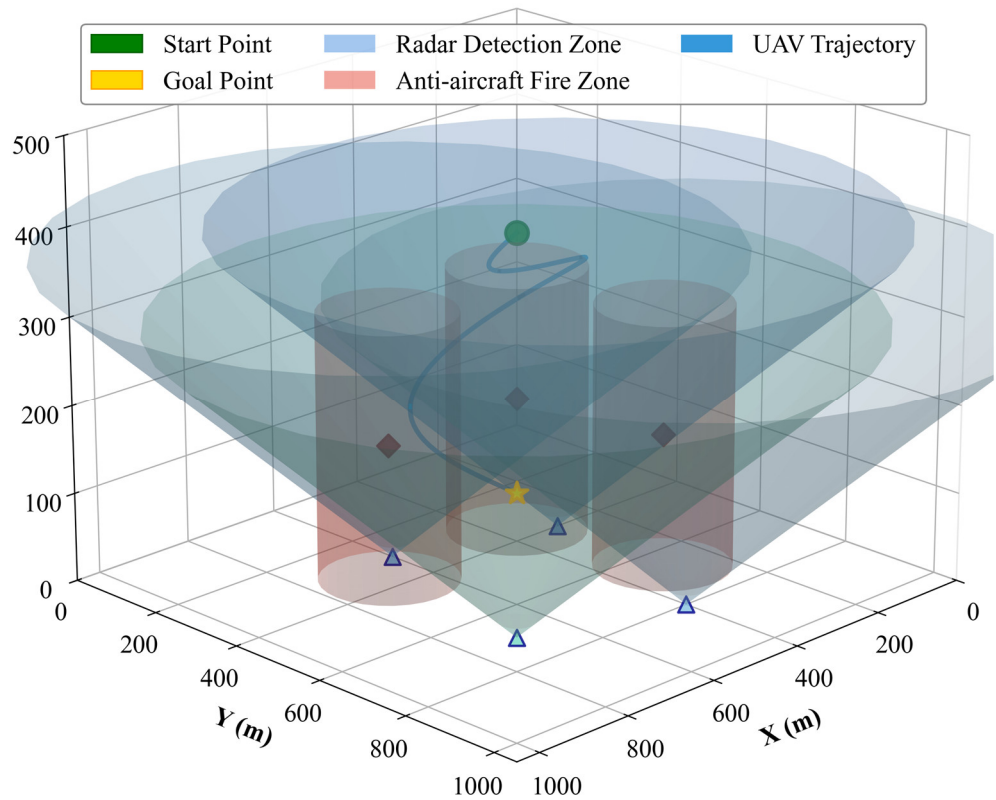


Figure 1. Simulation environment and threat scenario illustration.

The threat distribution strategy uses three placement algorithms to simulate threats distributed uniformly over the baseline environment, threats distributed in clusters that represent concentrated defense networks, and threats with random placement that ensure no overlap among threats. Each threat entity has its operational parameters, such as refresh rates for radar detection and response times within fire zones. The simulation environment allows dynamic activation of threats that can be activated or turned off during the simulation to reflect reality. The threat parameters in Table 1 are configured based on typical short-range air defense systems reported in open literature, with detection ranges and lethal radii representing common surface-to-air missile and anti-aircraft artillery characteristics.

Table 1. Threat environment parameters.

Parameter Description	Variable Name	Value
Maximum detection range (radar)	R_d	200–300 m
Detection angle	θ_d	60–90°
Baseline detection probability	P_0	0.7–0.9
Attenuation coefficient	α	0.01–0.02
Lethal radius (fire zone)	R_l	50–80 m
Effective radius (fire zone)	R_e	100–150 m
Number of threats (low density)	-	8–12
Number of threats (medium density)	-	13–19
Number of threats (high density)	-	20–25
Minimum threat separation	-	100 m
Threat types	-	Radar, Fire zone

2.1.2. UAV Dynamics Model

The dynamics of the UAV are represented by a simplified kinematic model that addresses the most significant constraints on motions. The state evolution follows the discrete-time equations:

$$\begin{aligned} x_{t+1} &= x_t + v_t \cos(\psi_t)\cos(\gamma_t)\Delta t \\ y_{t+1} &= y_t + v_t \sin(\psi_t)\cos(\gamma_t)\Delta t \end{aligned} \quad (1)$$

where (x, y) represent position coordinates on the horizontal plane, v denotes velocity, and ψ is the heading angle. The velocity and heading angle are updated based on the discrete actions selected by the DQN agent at each time step. Specifically, velocity changes by ±2 m/s for acceleration/deceleration actions within the bounds [v_{min}, v_{max}], and heading angle changes by ±15° for turning actions, as detailed in Section 2.2.3. The simulation model accounts for physical constraints, such as the lowest possible velocity $v_{min} = 15\ \mathrm{m/s}$, with the highest possible velocity set to $v_{max} = 35\ \mathrm{m/s}$ to emulate those of a fixed-wing UAV. The rates are constrained in terms of turning rates to ensure that the change in the heading direction $|\Delta\psi|$ per time step does not exceed 30°.

2.1.3. Training and Testing Scenario Generation

The environment generation pipeline ensures creation of diverse training and testing environments by incorporating various randomized processes. The need for diverse environment exposure to ensure robust learning in novel environment settings was highlighted by Wu et al. as inspiration behind the diversity requirements outlined here [17]. The training environments are generated procedurally with threat numbers varied randomly between 8 and 25 within the operational area with minimum 100 m separations among consecutive threats. Each generated environment defines start and end positions on opposite edges with randomly positioned targets within 150 m radius areas to counter overfitting to defined target position values. The environment difficulty level scoring employs an aggregate measure that considers threat density in threats per cubic kilometer area, average corridor width measures between threats, and shortest safe pathway length values. The lowest difficulty environments are defined with 8–12 threats and corridors above 200 m width, medium difficulty scenarios include 13–19 threats with 100–200 m corridor width values, and the most difficult settings include 20–25 threats with corridors below 100 m width. The set includes 5000 environment instances with training instances, 500 tuning environments with validation instances to adjust all parameters correctly without overfitting to various values during hyperparameter search, and 1000 testing instances distributed among difficulty settings without overlap with training environments to prevent biased generalizations. Each environment generation process includes assurance checks to ensure the existence of at least one possible pathway among threats with simplified RRT validation to exclude unsolvable instances. The UAV dynamics model parameters used in the simulation are summarized in Table 2.

Table 2. UAV dynamics model parameters.

Parameter Description	Variable Name	Value
Minimum velocity	v_{min}	15 m/s
Maximum velocity	v_{max}	35 m/s
Maximum heading angle change rate	$\lvert\Delta\psi\rvert$	30°/step
Time step	Δt	0.5 s
State dimension	-	53
Action space size	-	9 actions
Number of nearest threats tracked	k	8

2.2. Problem Formulation

2.2.1. Markov Decision Process Formalization

The UAV penetration mission problem is modeled as that of a Markov Decision Process (MDP), denoted as (S, A, P, R, γ), with state space S, action space A, transition probability function P:s × a→s″ ∈ [0,1], reward function R:s→a→r ∈ ℝ, and discount factor γ ∈ [0, 1]. At each stage t, the agent perceives the current state at ∈ S; chooses an action at ∈A according to its policy π; moves to the next state st + 1, conditioned on the current state-action pair (st, at), with dynamics P(st + 1 | st, at); and earns one step reward scalar signal rt = R(st, at). The task is to identify an optimal strategy π* that maximizes the infinite-horizon discounted cumulative return $J(\gamma, r_t) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$. The Markov property posits that the state transition probabilities are independent of past histories (states), with the current state and action fully determining the state transition process; this embedding holds if it is guaranteed that the employed state encodes all the information needed to make decisions about the next state-action pair taken by the UAV. The two types of termination are defined by mission accomplishment (reaching the GOAL) and mission failure (collision with threats or time-out).

2.2.2. State Space Design

The state space combines these three types of information to enable complete situational understanding. The state of the UAV consists of its position parameters (x, y), velocity magnitude v, and orientation angle ψ; these parameters constitute a four-dimensional vector that fully defines the kinematic configuration in the 2D plane. The environment perception state encodes threat information in an organized manner, with each threat represented by its relative position vector to the UAV, threat type (either radar or fire zone), lethal/detection radius, and current threat level. To standardize the state dimensions irrespective of the current number of threats, the k-nearest threats are extracted and ordered by distance, with k set to 8. This truncation is justified by two observations: (1) in penetration missions, immediate threats dominate action selection since distant threats allow sufficient reaction time for future replanning, and (2) experimental validation in high-density scenarios (20–30 threats) confirms that k = 8 captures sufficient information, as threats beyond this range typically lie outside the UAV's effective maneuvering horizon given kinematic constraints. While this may not fully preserve the Markov property in extreme cases, the 87.6% success rate in high-threat environments suggests the approximation remains effective. Xu et al. proved that state-detection features improve substantially the efficacy of path planning algorithms within complex environments; such evidence validates the incorporation of threat state features within the observation state space [18]. The relative goal state comprises Euclidean distance to the target location, azimuth angle to the current UAV position relative to the location, and elevation angle to the target location. Other features added to the state include the time budget remaining within the mission and violation flags that are binary values to indicate constraint violation within the past mission time budget. The state encodes 53 dimensions that are balanced to reduce redundancy while maintaining computational efficacy within state-of-the-art MLP models.

2.2.3. Action Space Design

The action space is modeled with a discrete action set consisting of nine actions to efficiently approximate the action-value function. The action space is defined as a discrete set consisting of nine actions combining velocity and heading maneuvers. The action set consists of combinations of velocity actions to either maintain the current velocity,

accelerate, or decelerate (three velocity choices) with heading actions to either turn left 15°, maintain heading, or turn right 15° (three choices), resulting in 3 × 3 = 9 possible action combinations. This 2D action space provides efficient approximation of the continuous action space while maintaining computational tractability for value function learning. The use of discrete action space provides efficiency over continuous action space, with the advantage that DQN's algorithm can efficiently approximate the action-value function without the need to explore the action space defined by continuous action parameters. The 15° heading increment and ±2 m/s velocity change were determined through preliminary experiments, where finer granularity (e.g., 5°) showed marginal performance improvement but significantly increased convergence time, while coarser granularity (e.g., 30°) resulted in frequent corridor navigation failures. The 0.5 s time step combined with 15° turning allows the UAV to achieve a minimum turning radius of approximately 85 m at average velocity, which is sufficient to navigate corridors above 100 m width as defined in our scenario configuration. Failure analysis in Section 3 confirms that most failures occur in extreme high-density scenarios with corridors below 100 m, indicating the action granularity limitation. The action constraints are managed with the use of masked actions that remove any actions that might violate the current state conditions that relate to the action to be taken; that is, the agent will not consider an action if that action has impossible parameters that might violate boundaries immediately after takeoff.

2.2.4. Reward Function Design

The reward function combines various objectives by means of weighted summation over four sub-functions that drive the agent to successfully execute penetration maneuvers. The task-oriented reward function provides positive reinforcement according to the task accomplishment level with values calculated as $r_{\text{goal}} = \beta_{\text{goal}} \cdot (d_{\text{prev}} - d_{\text{curr}})$, where d_{prev} and d_{curr} denote the distances from the UAV to the goal at the previous and current time steps, respectively, and β_{goal} is the goal progress weight coefficient. The parameter βgoal = 10 weighs the task accomplishment process. A giant final task accomplishment reward of 1000 is earned to assure rigorous task accomplishment. The threat-avoidance task has offset values defined by exponential functions that infer punishment corresponding to threats with values calculated as $r_{\text{threat}} = -\beta_{\text{threat}} \cdot \sum_i \exp\left(-\frac{d_i}{R_i}\right)$, whereby di describes the distance to the i-th threat with Ri defined as an effective radius. The parameter βthreat = 50 weighs punishment values on safety measures. The exponential function is adopted because it models the realistic threat characteristics: the danger level increases rapidly as the UAV approaches the threat center while providing smooth gradients that facilitate stable policy learning. This formulation aligns with the physical properties of radar detection and weapon engagement zones, where threat intensity grows nonlinearly with proximity. These functions emphatically punish momentum to threats with significant safety boundaries. The negative punishment values −1 are added to each step to ensure timely shortest action targets that save time. The violation punishment terms include massive negative punishment values with −500 assigned to threats, −300 to boundaries, and −200 to movement terms within the kinematic constraint violation term. The complete reward function parameters are summarized in Table 3.

Table 3. Reward function parameters.

Parameter Description	Variable Name	Value
Goal progress reward weight	β_{goal}	10
Goal achievement reward	-	+1000
Threat-avoidance penalty weight	β_{threat}	50
Time step penalty	-	−1
Collision penalty	-	−500
Boundary violation penalty	-	−300
Kinematic constraint violation penalty	-	−200

In summary, the UAV penetration path planning problem is formally defined as finding an optimal policy π^* that maximizes the cumulative discounted reward while navigating through threat-dense environments, subject to kinematic constraints and real-time computational requirements.

2.3. Multi-Head Attention-Enhanced DQN Framework

2.3.1. Overall Architecture Design

To address the above problem, we propose a multi-head attention-enhanced DQN architecture. The multi-head attention mechanism addresses the challenge of processing complex threat information by enabling selective focus on critical environmental features, while the dynamic priority replay addresses the sparse reward challenge by prioritizing informative experiences during training. The selection of DQN as the base framework is motivated by several considerations specific to penetration missions. First, the discrete action space formulation aligns naturally with value-based methods, enabling efficient Q-value estimation without continuous action optimization. Second, DQN's off-policy nature permits integration with prioritized experience replay, which is critical for learning from sparse successful penetration experiences. Third, compared to policy gradient methods such as PPO that require on-policy samples, DQN achieves superior sample efficiency in sparse-reward environments where successful trajectories are rare during early training stages. As shown in Figure 2 below, the architecture adopted an end-to-end learning process starting from raw state observations to hierarchical action-value estimation utilizing neural network layers. In this figure, the left side illustrates the feature extraction and multi-head attention modules, while the right side shows the value estimation network and training loop. Arrows indicate the data flow direction, and dashed lines represent the periodic target network synchronization. The architecture consists of four key layers operating in sequence to facilitate end-to-end learning: the first layer is the feature extraction layer that encodes diverse state data to shared representations. The subsequent layer incorporates the multi-head attention layer that discriminates key threat features within the data, utilizing multiple attention processes concurrently. The third layer is the value function estimation layer that combines features to approximate action values.

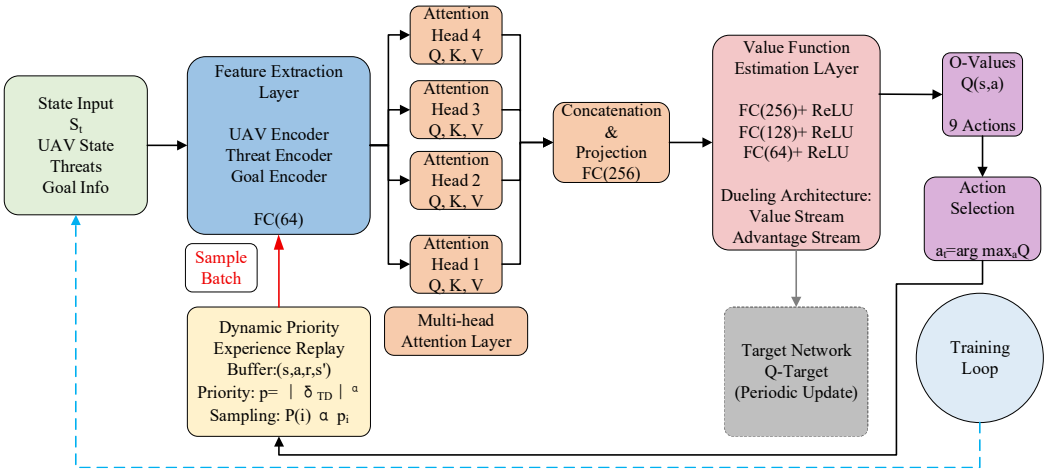


Figure 2. Multi-head attention-enhanced DQN architecture.

The modular architecture enables simultaneous optimization of each module without disrupting end-to-end differentiability during gradient-based learning. The online network engages actively with the environment to produce experiences, while the target network with synchronized parameters ensures stable Q-values during learning, as per the standard DQN routine. The dynamic priority experience replay module stores transitions in the prioritized experience replay buffer with dynamically updated priority values according to temporal difference errors to give higher importance to informative experiences. The architecture proposed here deviates from traditional DQN approaches by incorporating feature weights through the attention mechanism to allow the agent to prioritize its computations on significant aspects of the environment instead of equal consideration to each observation. The proposed architecture offers three key advantages: (1) parallel attention heads enable simultaneous extraction of diverse threat features, including proximity, lethality, and spatial density; (2) the learned attention weights provide interpretability for mission analysis; and (3) the modular design allows flexible adaptation to different threat configurations without retraining the entire network. The impact of enhanced attention mechanisms on decision-making in complex scheduling tasks was shown to be substantial by Chen et al. to include multi-head attention within the value network architecture in path planning tasks [19].

The flow of information is as follows: at each step in time, the current state observation is passed to the feature extraction layer to produce a context-informed representation. The representation is then passed to the multi-head attention layer to compute multiple attention heads that calculate relevance weights to threat features separately. The results are then concatenated to produce outputs that are passed to the value function estimation layer to produce Q-values corresponding to each action. During training, each transition is stored in the experience replay buffer with initial priority set to the corresponding TD error. The sampling process involves selecting samples with probability that is proportional to priority, with gradient updates to parameters with respect to TD error. The parameters of the target network are updated by periodically copying them from the online network to improve the stability of training.

2.3.2. Network Structure

The feature extraction layer uses a multi-branch architecture to extract features from different state components with custom encoders. The UAV internal state vector is passed through a two-layer fully connected network with ReLU activation functions that map the six-dimensional kinematic vector to 64 dimensions. The threat information is processed separately with each of the k-nearest threats mapped to an embedding using a shared

three-layer perceptron with 32, 64, and 64 units, respectively. The aim here is to share parameters among threats to guarantee parity during feature extraction. The result is that the process is permutation invariant. The goal relative information is mapped to an embedding with a two-layer network with 32 and 64 units, respectively. The embeddings are combined to form a complete feature vector with dimensions 640 = 64 + 8 × 64 + 64 to feed the attention module.

The attention layer consists of a multi-head attention mechanism with four fully parallel attention heads that process the threat embeddings separately. Each head discovers different types of relevance relationships among the threats by learning projections of the threat features into different query, key, and value subspaces. The UAV state embedding is used as the query vector, and there is one key and one value embedding per threat. The attention weights for head h are computed as $\alpha_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d_k}}\right)$, with Q_h and K_h denoting the projected query and key matrices and dk set to 64 dimensions to define the key dimension. The weighted threat representation for each head is then obtained as $V_h \alpha_h^T$, with V_h denoting the projected value vectors. The results are then combined elementwise and further transformed with a linear projection layer to obtain an attention-augmented feature vector with 256 dimensions. The multi-head mechanism allows various types of threats to be captured through attention concurrently, like those depending on proximity and those depending on lethality.

The estimation of the value function comprises three fully connected layers with 256, 128, and 64 units, respectively, each with ReLU activation functions and 0.2 dropout to prevent overfitting. The last layer produces the Q-value estimates for all nine discrete actions without activation to ensure unconstrained estimation of the values. The dueling network is integrated within the architecture by dividing the penultimate fully connected layer into two independent layers; one computes the value function V(s), which has one output neuron to represent the state value V(s), while the other computes the advantage functions A(s,a), with one output neuron for each action to calculate the advantage functions A(s,a) corresponding to each action. The final Q-value estimation is computed as Q(s,a) = V(s) + [A(s,a) − meana A(s,a)], which promotes more stable learning by separating state values from action advantages. The dueling network performs well with domains with similar outcomes arising from action space decisions; such environments are common during path planning tasks to move past threats in similar maneuvers. The detailed network architecture parameters are presented in Table 4.

Table 4. Network architecture parameters.

Parameter Description	Variable Name	Value
UAV state encoder dimensions	-	6 → 64
Threat encoder dimensions	-	32 → 64 → 64
Goal encoder dimensions	-	32 → 64
Number of attention heads	H	4
Key/value dimension per head	d=k = d_v	64
Multi-head output dimension	-	256
Value network hidden layers	-	256 → 128 → 64
Dropout rate	-	0.2
Output actions	-	9

2.4. Multi-Head Attention Mechanism

2.4.1. Attention Mechanism Principles

The attention mechanism is built upon the underlying Query Key-Value (QKV) formula, which allows for informative retrieval by virtue of relevance mapping. In terms of UAV path planning tasks, the query corresponds to the current decision state that needs to be analyzed in terms of the current environment context, while the keys correspond to the discriminative features of threats in the environment, and values are comprised of the real threat data to be sought. The formula underlying the attention mechanism begins with mapping the features to the queries and keys as follows:

$$\begin{aligned} Q &= W_Q x_{\text{query}} \\ K &= W_K X_{\text{context}} \\ V &= W_V X_{\text{context}} \end{aligned} \tag{2}$$

The attention weights measure how strongly each element in the context is related to the query with respect to a key-query/key pair compatibility function. The scaled dot product attention computes these weights as $\alpha = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$, with the scaling term $\sqrt{d_k}$ to prevent gradient saturation in the softmax function if the key dimension dk is large. The softmax function normalizes the attention weights to ensure that they form a valid probability distribution over the context elements. The attention output computes the weighted average over the vectors V to extract features with respect to attention weights α. The attention function computes Attention $(Q, K, V) = \alpha V = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$. The attention module provides the capability to dynamically weigh the most critical threats depending on the current state of the UAV instead of giving equivalent importance to all factors in the environment. The learned projections facilitate the discovery of abstract relationships between threats that are strategically positioned rather than physically nearby.

2.4.2. Multi-Head Attention Design

The multi-head attention is an extension of the standard attention mechanism. In the multi-head attention module, multiple attention processes are carried out concurrently in different subspaces. As shown in Figure 3 below, there are four attention heads to strike a balance between capacity and efficiency. Figure 3a illustrates the single attention head computation showing the scaled dot-product attention process, while Figure 3b demonstrates how four parallel heads are combined through concatenation and linear projection. The use of multiple attention heads allows various aspects of threats to be attended to concurrently without increasing the parameters excessively. Each attention head is allowed to have its set of projection matrices W_Q^h, W_K^h, W_V^h to facilitate computing separate attention outputs $\text{head}_h = \text{Attention}(QW_Q^h, KW_K^h, VW_V^h)$.

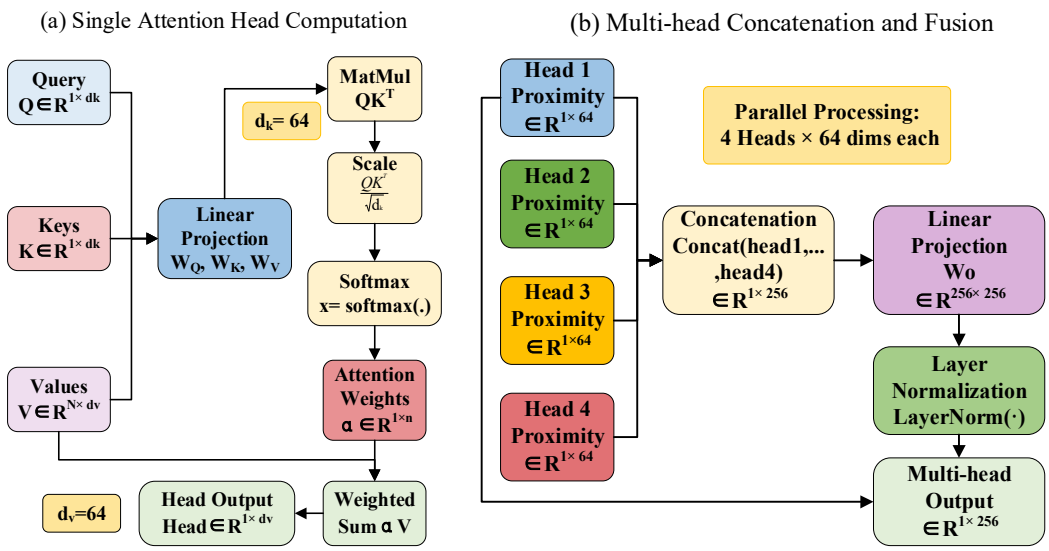


Figure 3. Multi-head attention mechanism detailed structure.

The four heads are specialized to capture different aspects of threats. The first head is normally learned to capture proximity attention, with heavy emphasis on nearby threats that need to be avoided immediately. The second head centers on high-threat targets, giving higher priority to dangerous fire zones rather than harmless obstacles, irrespective of distance. The third head attends to threats that are positioned on the directed pathway to the end zone to identify targets that lie on the best possible pathway. The fourth head captures threat density to spot areas with superimposed regions of detection that form dangerous passages. The individual tasks are realized without manual programming to enable the decomposed functions to emerge during training.

2.4.3. Feature Fusion Strategy

The outputs from individual attention heads are integrated through a two-stage fusion process that combines diverse perspectives into a unified representation. The initial concatenation operation stacks all head outputs along the feature dimension:

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \mathrm{head}_2, \ldots, \mathrm{head}_H) \tag{3}$$

where H = 4 denotes the number of heads. This concatenation preserves all information from individual heads, creating a high-dimensional representation of dimension H × dv, where dv is the value dimension per head set to 64, yielding a 256-dimensional concatenated vector. The concatenated features undergo linear transformation through a learned projection matrix W_O : $\mathrm{Output} = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)W_O$, which reduces dimensionality while learning to weight and combine information from different heads adaptively.

The projection layer serves multiple purposes beyond dimensionality reduction. It enables the network to learn complementary relationships between heads, potentially amplifying consensus signals when multiple heads agree on threat significance while downweighting contradictory signals. The trainable projection weights allow dynamic adjustment of head importance based on task requirements, effectively performing learned feature selection at the head level. Layer normalization follows the projection operation to stabilize the feature distribution and facilitate gradient flow through the deep network. This normalized output feeds directly into the value function estimation layers, maintaining end-to-end differentiability throughout the architecture. The proposed fusion strategy offers two advantages: (1) the concatenation operation preserves complete information

from all attention heads without loss, and (2) the learnable projection matrix adaptively weights head contributions based on task requirements rather than fixed manual weighting.

2.4.4. Integration with DQN

The multi-head attention module is integrated with the DQN architecture to improve its state representation prior to the estimation of Q-values instead of modifying the fundamental architecture formed by the value function network. The process involves feature-level integration with the standard DQN architecture, incorporating the result of threat information processed with attention mechanisms with the state embedding and goal state to produce a more robust state representation. The fundamental training process within the standard DQN architecture is maintained without any modifications to facilitate experience replay and target network updates. The capacity within the network to represent potential solutions has been further improved with the inclusion of attention mechanisms. The parameters within the attention module are updated during backpropagation with the temporal difference error to ensure that the attention module addresses accuracy within the estimation of the Q-value function instead of mere feature representation that focuses on penetration path planning tasks.

2.5. Dynamic Priority Experience Replay

2.5.1. Experience Replay Mechanism

Replay experience is one of the most essential techniques in deep reinforcement learning that helps reduce the temporal correlation among successive experiences in the training process, thus making better use of past experiences due to data reuse. The uniform sampling method selects transitions randomly from the replay buffer with the same probability, meaning that it assigns the same priority to each experience in the replay buffer. This uniform sampling faces challenges in complex environments due to the infrequent occurrence of informative experiences that are opportunities to gain significant learning in such environments. In penetration environments, experiences such as close calls with threats and navigating narrow corridors successfully are less frequent but more informative that need to be revisited more often.

The prioritized experience replay algorithm overcomes the above limitation by assigning transition weights according to learning values and sampling transitions proportionally to these values. The reasoning behind prioritized experience replay lies in the fact that transitions with higher temporal difference error values correspond to higher variations between current approximations to the values and realized returns, indicating areas that require more learning. The agent prioritizes transitions with higher error values to reduce estimation mistakes during learning and improve convergence to accurate values. Gok proved that the prioritized replay version of dueling double deep Q-networks greatly improves path planning tasks [20]. This is particularly useful during penetration operations due to the existence of sparse rewards and large state spaces that introduce large variations to transition informativeness.

2.5.2. Priority Calculation Method

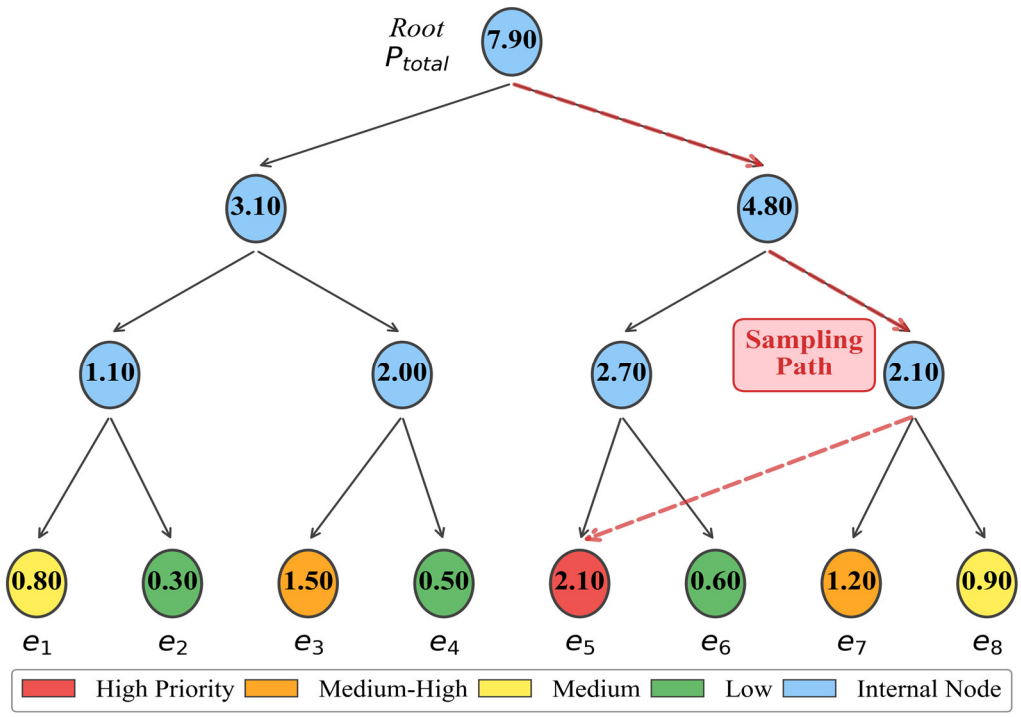
The priority assigned to each transition is the measure of the transition's potential contribution to learning progress and is reflected by the magnitude of the temporal difference error. When the transition (s_t, a_t, r_t, s_{t+1}) is added to the replay buffer, the temporal difference error is calculated as $\delta_t = | r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a') - Q(s_t, a_t) |$. The error provides direct insight into how surprising the transition is with respect to the

current policy; the higher the error values indicate, the greater potential there is to improve the value function with gradient updates to that transition.

The priority calculation involves this TD error in an exponential form to produce a probability distribution over which sampling is performed. The function $p_t = |\delta_t + \varepsilon|^{\alpha}$, with $\alpha \in [0, 1]$, is defined to ensure the priority has no zero values with the inclusion of the parameter $\varepsilon = 10^{-6}$, with α acting to allow a balance between prioritization and normal sampling. The values $\alpha = 0$ indicate uniform sampling, while $\alpha = 1$ illustrates full prioritization. The paper chooses $\alpha = 0.6$ to allow validation and prioritization among different learning processes among models. The values are updated after each transition with the function $p_t^{\text{new}} = |\delta_t^{\text{new}} + \varepsilon|^{\alpha}$; here, δ_t^{new} illustrates the updated form of the error term after each transition.

2.5.3. Sample Sampling Strategy

The implementation of prioritized sampling uses the sum tree data structure that facilitates efficiency in sampling proportionate to priorities in logarithmic time complexity. As shown in Figure 4 below, the sum tree is structured with the use of a binary tree to contain the transition priorities in leaf nodes and the cumulative values in parent nodes. The root node thus stores the total priority sum $P_{\text{total}} = \sum_i p_i$, enabling rapid sampling by generating a random value $v \sim \text{Uniform}(0, P_{\text{total}})$ and traversing the tree to find the leaf whose cumulative priority interval contains v. This mechanism provides two advantages: (1) the sum tree structure enables O(log N) sampling complexity compared to O(N) for naïve implementations, and (2) importance sampling weights correct the bias introduced by non-uniform sampling to ensure convergence guarantees.



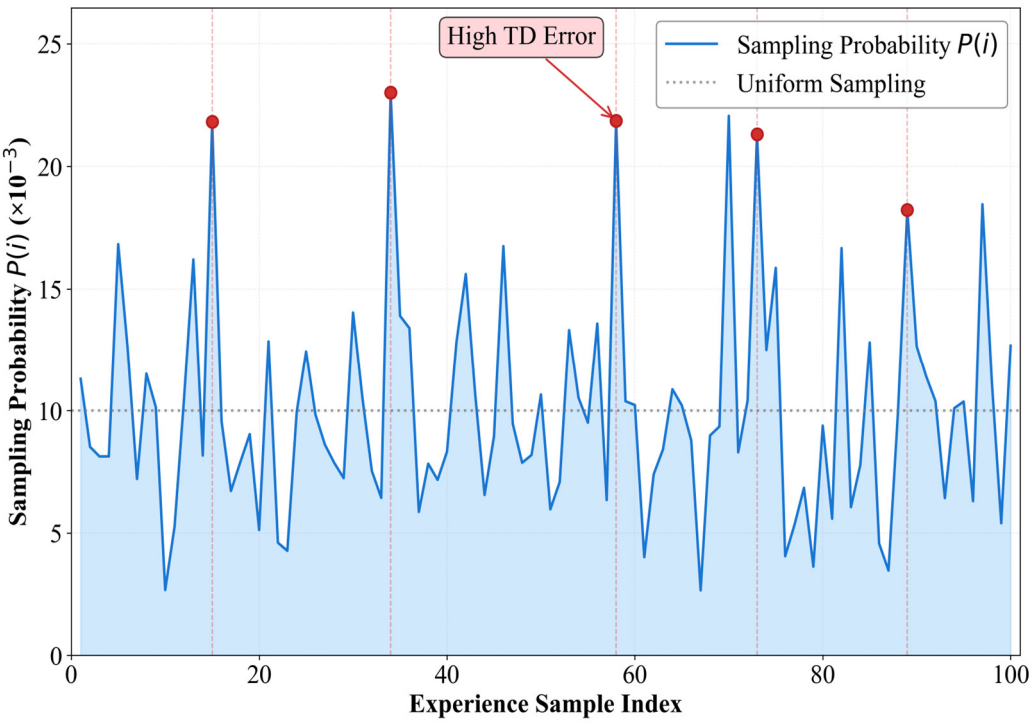


Figure 4. Dynamic priority sampling mechanism.

The transition sampling probability pi is defined by $P(i) = \frac{p_i}{\sum_k p_k}$, such that prioritized experiences are more likely to be seen during training. The biased sampling causes biased updates to the values function because the sampling distribution has a different expectation than the true distribution over seen transitions. Importance sampling weights are then used to mitigate the sampling bias by assigning each gradient step an importance weight that is inversely proportional to the sampling probability $w_i = (N \cdot P(i))^{-\beta}$, with N indicating the buffer size and the parameter taking values within [0, 1] to adjust the severity of the bias correction. The scheduler sets the annealing parameter beta to start with 0.4 and increase linearly to 1.0 over the initial half of the training process.

The updated gradient incorporates these importance weights through scaling the TD error during the backpropagation step:

$$L(\theta) = \mathbb{E}_i\left[w_i \cdot \left(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a; \theta) \right)^2 \right] \tag{4}$$

where w_i is the importance sampling weight, r is the immediate reward, γ is the discount factor, and θ represents the network parameters.

The use of importance weights ensures that transitions that are oversampled with higher priority are correspondingly less weighted each time they occur. The scaling is normalized by maxi wi to ensure that no weight becomes unreasonably large during training.

2.5.4. Priority Dynamic Adjustment Mechanism

The dynamic adjustment mechanism adjusts the strategy of prioritization during the training process to correspond to the learning demands that change with the stages of training. In the initial stage of exploration, since there are significant estimation errors in the value function globally, the use of extreme prioritization (α = 0.6) is appropriate to

eliminate significant misunderstandings and obtain an approximate understanding of the values. During later stages of training, the use of extreme prioritization might result in overfitting to transitions with unusual values.

The dynamic adjustment puts in place a priority decay process that ensures unusually high transition priorities are moderated to prevent unlimited dominance of transitions in the sampling distribution. Transitions that take on unusually high priorities above three standard deviations above the mean buffer priority undergo multiplicative priority decay $p_i \leftarrow 0.95 \cdot p_i$ in every update cycle until normalized to within normal priority levels. The decay factor 0.95 was empirically selected to ensure gradual reduction while maintaining relative priority ordering: at this rate, an outlier transition requires approximately 14 decay cycles to reduce by 50%, preserving sampling likelihood for genuinely important rare transitions. This approach maintains the convergence properties of prioritized replay since the importance sampling correction (Equation (4)) compensates for any sampling distribution changes, and the decay only affects statistical outliers rather than systematically important high-TD-error transitions whose priorities are continuously refreshed through replay. The process ensures that there are no pathological processes that lead to the monopoly of transitions in sampling probability distributions, such that there is enough diversity in every training batch to ensure that there is enough generalization capacity. Newly added transitions to the queue are assigned initial priorities set to be the current maximum priority in the buffer to ensure that each experience gets one opportunity within the sampling process before possible deprioritization due to true TD-error.

2.6. Training Algorithm

2.6.1. Algorithm Pseudocode

The pseudocode of the proposed algorithm is shown in Algorithm 1.

Algorithm 1. Pseudocode of the Multi-head Attention DQN with Dynamic Priority Experience Replay algorithm.

Step	Algorithm Description
1	Initialize online Q-network Q(s,a;θ) with multi-head attention module and target network Q(s,a;θ⁻)
2	Initialize prioritized replay buffer D with capacity N = 100,000
3	Set hyperparameters: learning rate $\alpha = 3 \times 10^{-4}$, γ = 0.99, α_priority = 0.6, β_start = 0.4, β_end = 1.0, ε_start = 1.0, ε_end = 0.05, batch_size = 64, C = 1000
4	for episode = 1 to E do
5	Reset environment, obtain initial state s_1
6	for t = 1 to T do
7	Select action: a_t = argmax_a Q(s_t, a; θ) with probability 1 − ε, else random action
8	Execute action a_t, observe reward r_t and next state s_{t + 1}
9	Store transition (s_t, a_t, r_t, s_{t + 1}) in D with initial priority p = max_i p_i
10	if |D| ≥ batch_size then
11	Sample mini-batch according to priority distribution: P(i) = p_i^α/Σ_k p_k^α
12	for each sampled transition i do
13	Compute TD target: y_i = r_i + γ max_{a'} Q(s'_i, a'; θ⁻)
14	Compute TD error: δ_i = y_i − Q(s_i, a_i; θ)
15	Update priority: p_i ← |δ_i| + ε
16	Compute importance weight: w_i = (N · P(i))^{−β}/max_j w_j
17	end for

```
                Update online network: θ ← θ − α∇_θ L(θ), where L(θ) = Σ_i w_i δ_i²
                Apply dynamic priority decay: if p_i > μ + 3σ then p_i ← 0.95p_i
                end if
                if t mod C = 0 then
                    Synchronize target network: θ⁻ ← θ
                end if
            end if
        end for
    end for
```

2.6.2. Hyperparameter Configuration

The choice of the hyperparameters achieves a tradeoff between learning rate, stability, and convergence accuracy by empirical testing. The learning rate α is assigned a value of 3×10^{-4} with the use of the Adam optimizer with momentum parameters set to the defaults $\beta_1 = 0.9$ and $\beta_2 = 0.999$. The learning rate has been set to be somewhat conservative to prevent learning from diverging due to the potential magnitude of parameter updates during learning. The discount factor $\gamma = 0.99$ assigns greater importance to the end returns that are more pertinent to penetration missions due to the potential absence of rewards within the environment.

The strategy for exploring the action space follows an ε-greedy choice with parameters starting with $\varepsilon_{start} = 1.0$ for initial aggressive exploration of the state-action space and then linearly decaying to the final value $\varepsilon_{end} = 0.05$ after 800 episodes. The remaining 5% exploration rate is then maintained to prevent the learning process from converging to suboptimal policies. The experience buffer has enough space to store N = 100,000 transitions in the state-action space and still be computationally efficient. The batch size is set to 64 samples.

The prioritized replay exponent $\alpha = 0.6$ determines the level of prioritization with the importance sampling correction exponent β annealing linearly from $\beta_{start} = 0.4$ to $\beta_{end} = 1.0$ over the first 1000 episodes. The multi-head attention architecture sets the key and value dimensions to $d_k = d_v = 64$ with four attention heads that are then combined to form 256 dimensions. The dropout rate is set to 0.2 after every layer within the value function estimation network is utilized to prevent overfitting. The target network is updated every C = 1000 time steps to ensure enough temporal stability is allowed for calculation of TD targets without the targets becoming stale.

2.6.3. Training Strategy

The training process is designed with various mechanisms that ensure strong convergence properties. Network synchronization carries out a hard update process that involves copying the parameter set from the online network over every C steps to the target network parameters. The process involves $\theta^- \leftarrow \theta$ instead of exponential moving averages to ensure that there are no temporal dependencies between value function estimation and targets.

Early stopping criteria include monitoring validation set performance to reduce overfitting and redundant computation. Every 50 training episodes, the process tests the current policy on a validation set with 50 holdout validation scenarios covering all levels of difficulty. If the validation success rate does not improve within 200 episodes, the training stops as learning with diminishing returns begins with the risk of overfitting to the training scenarios.

The training stopping condition assesses various criteria cumulatively, such as validation success rates above 95% within 100 consecutive validation attempts, reaching the episode cap of 3000 episodes, and validation stagnation that causes early stopping.

Furthermore, gradient clipping with a threshold set to 10 helps to mitigate problems with exploding gradients caused by instances with high priority due to large TD error values. The priority decay function is activated after 500 episodes to reduce values above three standard deviations from the mean to ensure that there are variations in sampling. The complete training hyperparameters are listed in Table 5.

Table 5. Training hyperparameters.

Parameter Description	Variable Name	Value
Learning rate	α	3×10^{-4}
Optimizer	-	Adam
Momentum parameters	β_1, β_2	0.9, 0.999
Discount factor	γ	0.99
Initial exploration rate	ε_{start}	1.0
Final exploration rate	ε_{end}	0.05
Exploration decay episodes	-	800
Experience buffer capacity	N	100,000
Mini-batch size	-	64
Priority exponent	α	0.6
Importance sampling (initial)	β_{start}	0.4
Importance sampling (final)	β_{end}	1.0
IS annealing episodes	-	1000
Target network update frequency	C	1000 steps
Gradient clipping threshold	-	10.0
Maximum training episodes	E	3000
Priority small constant	ε	10^{-6}

3. Results

3.1. Experimental Setup

3.1.1. Hardware and Software Environment

The experiments are carried out on a workstation with Intel-Core-i9-12900K CPU (16 cores, 3.2 GHz Base clock), 64 GB DDR4 RAM, and an NVIDIA GeForce RTX 3090 graphics card (24 GB Video RAM). The Software Stack includes the operating system "Ubuntu 20.04 LTS," "Python 3.8.10," "PyTorch 1.12.1 with CUDA 11.6" (a GPU acceleration library), "OpenAI Gym 0.21.0" (environment standardization library), NumPy 1.23.0 (a numerical computing library), Matplotlib 3.5.2 (data plotting library), and TensorBoard 2.9.1 (training monitoring library). The above setup was set to efficiently train multiple variants of the algorithm in parallel with each training task taking 18–24 h, depending on the convergence rates. The setups' random state was set to be uniform over all tasks to ensure that all tasks are repeated with the same setup to remove variability due to differing initial starting conditions through 10 independent training sessions with each algorithm configuration to capture variations in learning rates due to the stochastic process.

The simulation engine has been written in Python 3.8 with the backend environment consisting of PyTorch 1.12 as the deep learning engine and OpenAI Gym 0.21 as the environment standardizer. The environment has integrated NumPy as the numerical library and Matplotlib as the plotting library to ensure conformity with standard reinforcement learning development ecosystems. The spatial model uses a two-dimensional Cartesian coordinate space to simulate the operational environment. The environment has dimensions set to 1000 m × 1000 m, focusing on horizontal maneuvering, which is the primary consideration for path planning in penetration missions. This 2D simplification reduces computational complexity while maintaining the essential characteristics of threat

avoidance and path optimization. During these missions, an increase in degrees of freedom is realized with changes in altitude to evade threats. The environment has a discrete step time of 0.5 s to ensure that computations are efficient without losing realism in modeling flight dynamics. The simulation environment configuration parameters are detailed in Table 6.

Table 6. Simulation environment configuration parameters.

Parameter Description	Variable Name	Value
Environment dimensions (X × Y)	-	1000 m × 1000 m
Discrete time step	Δt	0.5 s
Programming language	-	Python 3.8
Deep learning framework	-	PyTorch 1.12
Environment interface	-	OpenAI Gym 0.21
Numerical library	-	NumPy 1.23
Visualization library	-	Matplotlib 3.5
Maximum episode length	T_{max}	2000 steps
Number of training scenarios	-	5000
Number of validation scenarios	-	500
Number of testing scenarios	-	1000

3.1.2. Baseline Algorithms

The proposed multi-head attention-enhanced DQN with dynamic priority replay was tested with respect to four existing baseline algorithms to ensure its efficacy. The baseline DQN was kept as the standard implementation with the original uniform experience replay and one target network to depict the fundamental value-based solution. The Double DQN (DDQN) overcame the estimation bias issue with action selection and evaluation decoupling, acting through the use of the online network to choose actions and the target network to compute values. The Duelling DQN furthered action values by decomposing the Q-value function with distinct networks to compute values and advantages. The Proximal Policy Optimization (PPO) functioned as the baseline method with the use of clipped surrogate objective and generalized advantage estimation as the policy gradient method. The baseline algorithms retained the same state space representation, action space components, and scoring functions to guarantee direct comparability after tuning distinct hyperparameters to optimize each algorithm's results. Network structures within baselines had the same capacity without incorporating attention, as in the proposed solution that has similar dimensions to decouple the effect of utilizing attention instead of focusing on mere capacity.

3.1.3. Evaluation Metrics

Performance evaluation utilized five diverse measures that captured various dimensions of path planning quality. The success rate was the mission accomplishment fraction that successfully reached the destination without touching threats and staying within boundaries to guarantee the efficacy of each learning policy. The average path length was a measure that defined the trajectory efficiency by calculating the Euclidean distance covered from the initial to destination point normalized by the direct distance to facilitate analysis within scenarios that differed in scale. The threat-exposure degree defined the safety margin in terms of cumulative normalized proximity to threats over the entire trajectory formula that was defined as $C = \sum_t e^{-d_t / R_{\text{avg}}}$, wherein dt shows the minimum threat distance and Ravg shows the average radius. A smaller degree provides safer trajectories that stay further away from threats. The other measures related to computing

time calculated the difference within training time that was defined as the entire CPU time to converge until completion and action-time that was defined as the reaction time that computes an action with state observation. Convergence time was defined as the number of training experiences until 90% validity was reached on final success rate within validation sets that represented learning efficacy and pace. The statistical analysis to validate significant results in each set was conducted with paired t-testing with Bonferroni correction that had multiple testing in each set with significance levels $p < 0.05$. To verify the t-test assumptions, we performed Shapiro–Wilk tests confirming approximate normality of sample means across 10 independent training runs and applied non-parametric Wilcoxon signed-rank tests as robustness checks. The significance conclusions remained consistent under both parametric and non-parametric methods, with all pairwise comparisons maintaining $p < 0.01$ after Bonferroni correction, supporting the robustness of reported statistical claims.

3.1.4. Experimental Parameter Configuration

The protocol tested all algorithms with threats of varying densities corresponding to low (8–12 threats with average distances over 200 m), medium (13–19 threats with 100–200 m distances), and high (20–25 threats with densely packed formations and narrow corridors below 100 m) environment complexities. Each environment complexity was tested with 1000 randomly generated scenarios with variable threat location, types, and parameters within set ranges. The training of each algorithm had similar parameters, such as the episode count (maximum 3000 episodes), testing frequencies (every 50 episodes), and validation set sizes (50 scenarios per test). The proposed algorithm and baseline algorithms shared parameters such as batch sizes (64), buffer sizes (set to 100,000 values for experience replay buffers), and the discounting factor (set to 0.99). Additionally, the training process shared parameters such as the optimizer (Adam), gradient clipping values (set to 10.0), and learning rates with only algorithm-specific components (like attention layers and prioritized actions) differing between implementations.

3.2. Algorithm Performance Comparison

3.2.1. Training Curve Comparison

The learning dynamics among various algorithms portray undeniable variations within learning efficiency and convergence rates. Figure 5 illustrates the average cumulative reward during 3000 episodes of training to highlight that the proposed multi-head attention-reinforced DQN with dynamic priority replay learning better over the duration of training. The proposed model depicts faster learning processes with rapid increments in average cumulative rewards above 400 in the initial 500 episodes to significantly lead other algorithms. The baseline DQN depicts the lowest convergence rate with stagnation from episode 1800 to conclude with rewards close to 350, with only marginal increments by DDQN to offset overestimation biases to converge rewards to 380. The duel learning scheme with DQN manifests improved learning dynamics with faster acquisition of cumulative rewards above 420 by episode 2000 due to the decomposed architecture designed to ease state values learning. The PPO algorithm depicts stable learning with cumulative rewards above 400 after initial increments with smaller magnitudes from episode 1200 to account for potential overheads within its policy gradient learning process.

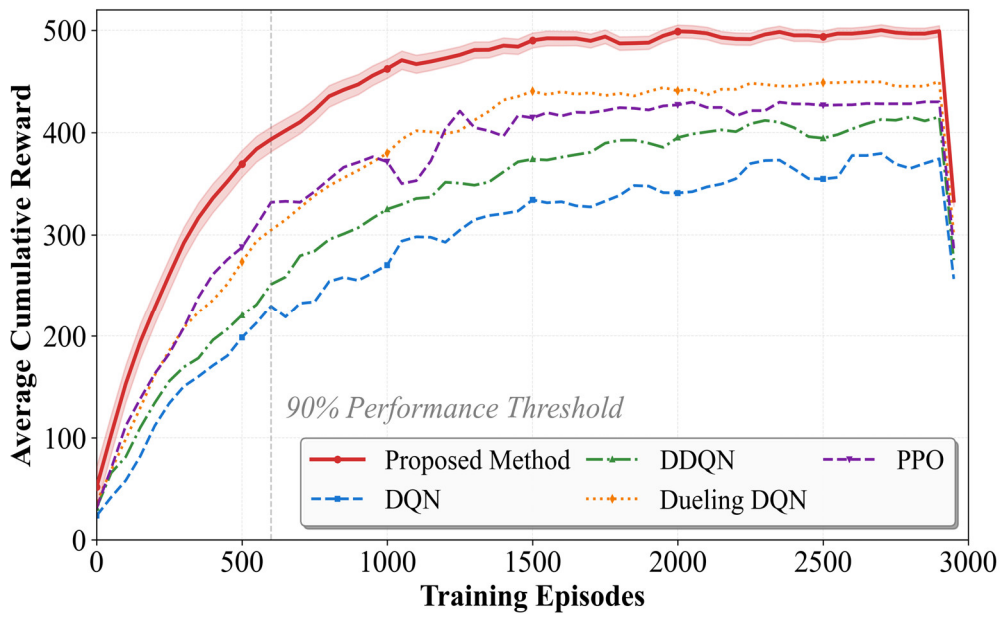


Figure 5. Training performance comparison across algorithms.

The learning curve of the proposed method shows much smoother learning with smaller variance, as denoted by the tight confidence interval shading during learning. This is due to the learning prioritization strategy focusing on informative experiences and the use of multi-head attention in learning robust features. The difference in rewards between the proposed method and the best baseline (Dueling DQN) is about 80 points, which shows that there is a 19% relative improvement. In terms of relative learning efficiency, it can be seen that 90% of the final rewards are reached within 600 steps by the proposed method while the baselines take 1200–1800 steps.

3.2.2. Quantitative Performance Comparison

Results on the test set of 1000 scenarios are shown in Figure 6. The proposed approach has a mission success rate of 94.3% ± 1.2%, which is substantially better than the baseline approaches (post hoc tests yield $p < 0.001$ for each pair). The standard DQN has only 76.8% ± 2.3% mission success rate, which clearly indicates that the uniform replaying and the standard attention are strongly suboptimal approaches. DDQN improves to 81.5% ± 1.9% through reduced overestimation, while Dueling DQN reaches 87.2% ± 1.6%, approaching but not matching the proposed method's performance. PPO achieves 83.7% ± 2.1% success rate with higher variance reflecting its on-policy nature's sensitivity to exploration quality.

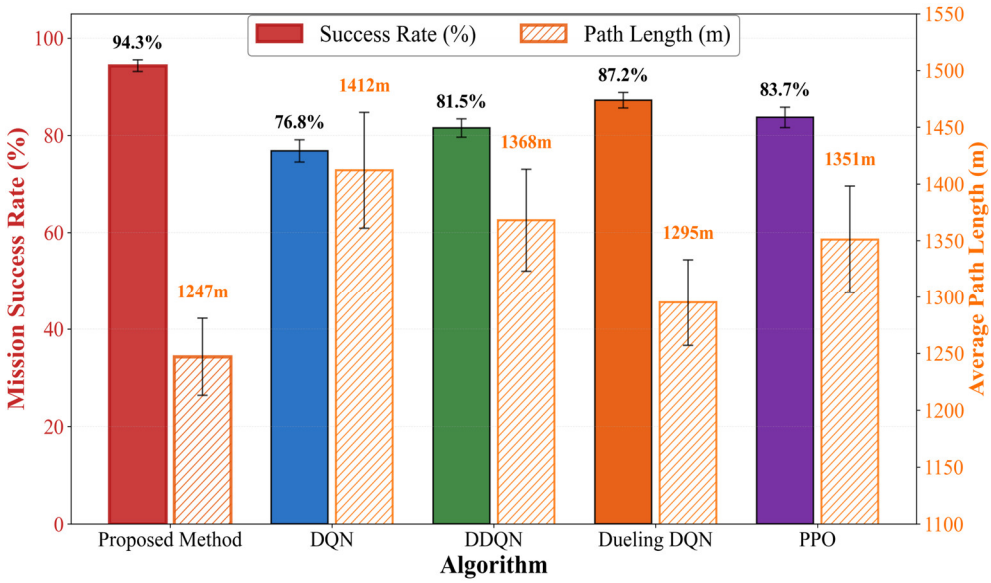


Figure 6. Quantitative performance comparison on test dataset.

Average path length results demonstrate the proposed method's trajectory efficiency, achieving 1247 ± 34 m compared to the straight-line distance baseline of 1131 m, representing only 10.3% excess distance. Baseline algorithms exhibit 15–25% excess distances, with DQN producing paths averaging 1412 ± 51 m due to inefficient threat avoidance maneuvers. Threat exposure degree measurements reveal the proposed method maintains superior safety margins with exposure index 0.12 ± 0.02, substantially lower than baseline values ranging from 0.19 to 0.28. Inference latency remains computationally tractable at 18.3 ± 1.1 milliseconds per action despite the additional attention computations, satisfying real-time requirements for UAV control systems operating at 10–20 Hz update rates.

3.2.3. Convergence Analysis

Convergence analysis determines learning efficiency by measuring the number of episodes to attain performance thresholds. The proposed approach converges to 90% of its end success rate in 612 ± 45 episodes, which shows 2.2× faster learning convergence compared to standard DQN with 1340 ± 89 episodes. The acceleration mainly comes from PR with secondary improvements brought about by the MA module that ensures better state representation. Convergence analysis evaluates the final 500 training episodes after convergence and finds that the proposed approach keeps the standard deviation of rewards to 12.4 instead of 23.7 with DQN and 19.8 with PPO. To verify robustness of the convergence metric, we examined alternative thresholds: the proposed method reaches 80%, 85%, and 95% of final performance at 485, 542, and 856 episodes, respectively, consistently outperforming baselines across all thresholds. Additionally, policy stability analysis measuring action consistency across similar states shows the proposed method achieves 94.2% action agreement compared to 87.3% for DQN, confirming that convergence benefits persist when examining policy stability rather than cumulative rewards alone.

3.2.4. Path Visualization Comparison

Representative qualitative comparisons among the paths highlight the distinct difference in behaviors delivered by algorithms. In medium-complexity tasks involving 15 threats, the proposed solution follows smooth trajectories that ensure consistent safety margins to detection zones with the least detours. The standard DQN shows irregular turning behavior and, at times, gets close to threats due to poor understanding of threats

and the accuracy of the value function. The DDQN and Dueling DQN show better trajectories but still experience poor decisions in routing, with instances of taking longer routes to circumnavigate corridors instead of threading through safe corridors. The PPO shows smoother trajectories but demonstrates poor understanding in selecting optimal corridors and instead chooses cautious corridors with more detours. The proposed solution shows intelligent prioritization in threats with higher margins to fire zones and gets closer to radar zones with lower threats due to the validity in the use of the multi-head attention approach to allow diverse reactions to threats.

3.3. Ablation Study

3.3.1. Multi-Head Attention Module Effectiveness

Ablation studies are conducted to analyze the effect of each component on the whole model. The effectiveness of the multi-head attention mechanism was tested by performing removal and adjustment tests. When comparing the full model with the version with attention components replaced with normal fully connected layers with the same amount of parameters, there is significant degradation in the no-attention setup with only 87.6% success rates compared to the full model with 94.3%. A degradation of 6.7% indicates that the attention mechanism contributes to the function beyond the capacity increase due to the parameter-balanced model that does not reach similar results with lower capacity.

The "single-head vs. multi-head" analysis evaluates if multiple attention heads in parallel work better as opposed to one attention mechanism with the same dimension. The "single-head" version with 256-dimensional output has the same combined multi-head dimension but only 89.4% success rate, suggesting that decomposing tasks functionally with separate attention entities emphasizing distinct threat features outperforms integrated attention. The study of the weighted attention features demonstrates such specialization, with distinct features attending to proximity, damage potential, trajectory alignment, and density features, respectively. The number of attention heads' analysis starting from one to eight identifies "four" to be optimal. "Two-head," "three-head," "four-head," "five-head," and "eight-head" settings obtain 90.8%, 92.5%, 94.3%, 94.1%, and 93.4%, respectively. The diminishing returns beyond four heads can be attributed to representational saturation: with only four distinct threat aspects to capture (proximity, lethality, alignment, and density), additional heads learn redundant or noisy features. Cross-head correlation analysis shows average pairwise correlation increases from 0.31 (four heads) to 0.52 (eight heads), indicating feature redundancy. Furthermore, increased parameter count without corresponding representational benefit may cause optimization difficulties and mild overfitting.

3.3.2. Dynamic Priority Mechanism Effectiveness

The contribution of the dynamic priority experience replay method was isolated by contrasting it with uniform sampling while ensuring the same architectures are used. The uniform sampling baseline has a success rate of 88.9%, which is 5.4% lower than the prioritized one; this verifies that giving higher priority to valuable experiences leads to faster learning. Beyond success rate improvements, prioritized replay reduces convergence time by 41%, requiring only 612 episodes to reach 90% of final performance compared to 1038 episodes for uniform sampling. This acceleration stems from more efficient utilization of informative transitions concentrated around decision boundaries and narrow corridors where small policy changes yield large performance impacts.

Comparison of priority calculation strategies evaluates alternative formulations for assigning transition importance. The adopted TD-error-based approach ($p = |\delta + \varepsilon|^{\alpha}$) outperforms alternatives, including loss-based priorities ($p = L(\theta)^{\alpha}$) achieving 91.7% success, recency-based weighting (p = age-α) reaching 89.3%, and random prioritization ($p \sim$

Uniform) attaining 88.5%. The TD-error formulation proves most effective because it directly measures learning potential through value function approximation errors, whereas loss-based methods conflate individual transition informativeness with batch-level training dynamics, and recency weighting arbitrarily emphasizes recent experiences regardless of actual learning value. Experiments with different priority exponents $\alpha \in \{0.4, 0.5, 0.6, 0.7, 0.8\}$ confirm optimal performance at $\alpha = 0.6$, balancing prioritization strength against sampling diversity.

3.3.3. Component Combination Analysis

Systematic evaluation of all component combinations quantifies individual and synergistic contributions through a 2 × 2 factorial design varying attention mechanism presence and priority replay activation. As illustrated in Table 7, the four configurations exhibit distinct performance profiles across multiple dimensions. The baseline configuration (no attention, uniform replay) represents standard DQN achieving 76.8% success rate with convergence at 1340 episodes and path length 1412 m. Adding only multi-head attention (attention present, uniform replay) improves success to 88.9% and reduces path length to 1298 m, demonstrating attention's value for state representation. Adding only dynamic priority (no attention, priority replay) elevates success to 87.6% with faster convergence at 892 episodes, confirming replay's learning acceleration. The complete configuration combining both mechanisms achieves 94.3% success, 612-episode convergence, and 1247 m paths, exhibiting superadditive performance gains exceeding the sum of individual contributions.

Table 7. Ablation study quantitative comparison.

Metric	Complete Model	w/o Priority	w/o Attention	Baseline
Success Rate (%)	94.3	88.9	87.6	76.8
Convergence (episodes)	612	1038	892	1340
Path Length (m)	1247	1298	1276	1412
Threat Exposure	0.12	0.18	0.21	0.28
Inference Time (ms)	18.3	18.0	14.7	14.7

The radar plot highlights the complementary strengths of various architectures with respect to five dimensions of evaluation scaled between 0 and 100. The multi-head attention module mainly contributes to better pathway quality (reducing length by 8.1% over baseline) and threat avoidance (improving safety by 31%), indicating its application to intelligent understanding. The dynamic priority module mainly decreases convergence time (improving by 33% over baseline) and promotes success rate (increasing by 10.8%), signifying learning efficiency improvement. Overall, the integrated model demonstrates balanced superiority over all dimensions. The radar plot shows extensive coverage over all dimensions compared to partial models that are strongly superior over isolated dimensions but weaker over others. The results are verified by two-way ANOVA tests that indicate significant results over the main factors attention (F-statistic = 127.3; p-value < 0.001) and priority (F-statistic = 98.6; p-value < 0.001), as well as over interaction (F-statistic = 23.4; p-value < 0.001), affirming the interactive process among components.

3.3.4. Reward Parameter Sensitivity Analysis

To verify the robustness of the learned policy to reward coefficient variations, we conducted sensitivity analysis by varying key parameters within ±50% of their default values. Results show that $\beta_{goal} \in [5, 15]$ and $\beta_{threat} \in [25, 75]$ maintain success rates above 90%, indicating reasonable tolerance to coefficient tuning. The collision penalty ranging

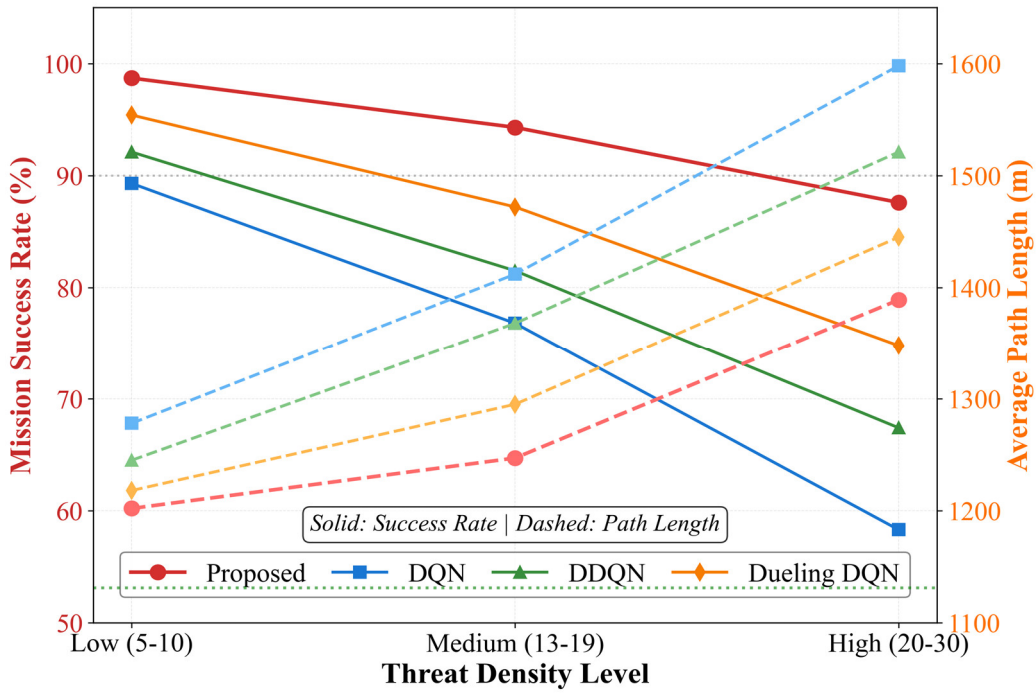
from −300 to −700 yields consistent avoidance behavior. However, reducing the goal achievement reward below +500 significantly degraded goal-reaching performance, suggesting this parameter is more critical. Overall, the policy demonstrates stability across moderate reward scaling variations, though extreme modifications may require retuning.

3.4. Complex Scenario Testing

3.4.1. Threat Density Variation Experiments

The overall evaluation with varying levels of threat density tests the readiness and versatility of the proposed solution to manage environments with differing degrees of complexity. In low-density settings with 5–10 threats, the environment was relatively conducive with numerous safe zones, and the proposed solution performs almost perfectly with 98.7% success and average path lengths that are no more than 6.2% greater than the direct path solution. The standard DQN solution has 89.3% success in these settings and proves that even in such benign environments with stringent requirements, such as accurate avoidance of threats, standard approaches are no straightforward task. The difference in performance between the proposed and standard approaches is still quite significant with 9.4% difference.

Medium-density settings with 13–19 threats correspond to the mainstream training set. As seen in Figure 7, the proposed solution preserves 94.3% success with rather stable average path lengths of 1247 m over the entire set of complex settings. The degradation in terms of success rate over low-density settings still stays within 4.4 percentage points, while the scaling is excellent. The baseline approaches are more vulnerable to the rise in threat density. Specifically, the success rate drops to 76.8% for DQN and 81.5% for DDQN with degradation of 13–20 percentage points over low-density settings. These results confirm that the proposed solution has better scaling properties to efficiently deal with complex threat environments due to superior feature analysis with the aid of attention and prioritized learning.



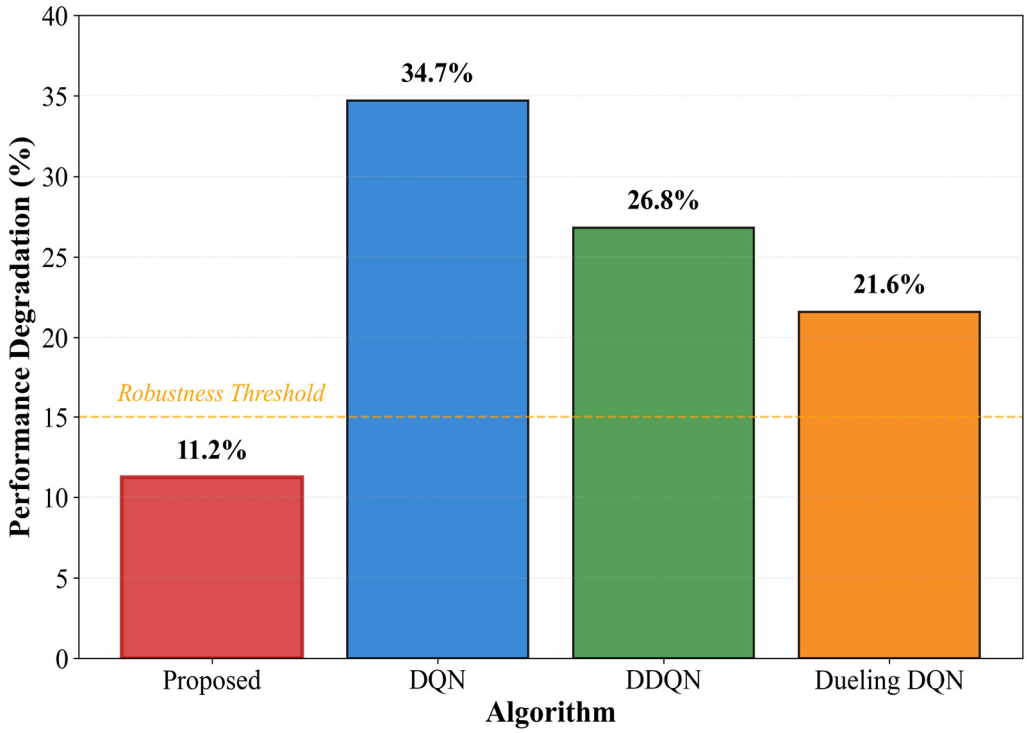


Figure 7. Threat density scaling analysis.

High-density environments with 20–30 threats are highly challenging operational settings with little free passage area and strong overlap among threats. The proposed solution has a success rate of 87.6% in these challenging settings, performing much better compared to baselines with success rates of 58.3% and 67.4% achieved by DQN and DDQN, respectively. The path length has increased to 1389 m, with 22.8% overhead over direct distance due to complex maneuvers to weave through tight corridors. Notably, the 11.1-point difference in success rates seen with higher density compared to medium density is much smaller compared to baselines with 18.5–25.2-point decreases to reflect the robustness gain with the proposed solution. The use of the multi-head attention module is most beneficial in high-density environments, where it becomes essential to consider nearby threats to make safe passage decisions.

3.4.2. Dynamic Environment Adaptability

Dynamic threat scenarios test the resilience of the algorithm to adaptive environments that are more complex than the static settings presented to the algorithm during training. In moving threats tests, three to five threats are moving with speeds ranging from 5–8 m/s according to set waypoints while threats are stationary. The suggested algorithm has a success rate of 89.7% in moving threats tests with more robust results since the success rate only dropped by 4.6% compared to stationary threats tests. In contrast to other algorithms, DQN has a lower success rate of 62.1%.

Sudden threat appearance scenarios mimic practical operational uncertainty with novel threats triggered in the middle of the mission. In these tests, two to four novel threats are randomly added to the mission after the agent has committed to acting on one trajectory. The proposed solution correctly adapts to threats 85.3% of the time, with average evasion action taken within three to five time steps after detection. The application of multi-head attention makes it possible to quickly assess threats. The PPO algorithm performs with 82.8% accuracy, thanks to its gradient-based policy that promotes more fluent behavior adjustment but still lags behind the proposed solution.

3.4.3. Generalization Capability Assessment

The zero-shot transfer learning tests the transferability of the learned policy to unseen environments without any further learning. When tested with novel maps that are similar in terms of density factors but with different threat distributions, the success rate achieved is 91.2%, which is only 3.1 percentage points lower than that in the usual environment settings, thus establishing the efficacy of learning the transferable policy. The baselines are more vulnerable to overfitting, with a decline in success rates by 8–15 percentage points on novel maps. To further verify that the policy does not overfit to the validation distribution, we tested on out-of-distribution scenarios, including clustered threat formations, linear barrier configurations, and asymmetric threat densities not present in training. The proposed method maintains 88.4% average success rate across these structured environments, compared to 91.2% on novel maps from similar distributions, demonstrating reasonable generalization beyond the predefined difficulty categories.

The map size variation tests check scale invariance properties by testing on environments ranging from 500 m × 500 m × 250 m to 1500 m × 1500 m × 750 m with similar scaling factors for threat densities. The proposed algorithm sustains 92–96% success rates over this threefold size change to show that the state representation's spatial reasoning about relative positions is scale-invariant and robust to scaling. Inference times are linear in episode length but stay within real-time boundaries with 18–22 ms per action overhead over all map sizes. The capacity to concentrate on $k = 8$ threats, irrespective of environment size, is due to the attention module. However, the scalability of $k = 8$ has limitations: preliminary tests with $k \in \{4, 8, 12, 16\}$ showed that $k = 8$ provides optimal tradeoff between performance and computational efficiency in our scenarios with up to 30 threats. In larger-scale contexts with threat swarms where numerous threats exert similar influence, the fixed $k = 8$ may lose critical information. Performance degradation was observed when threat density exceeded 40 threats with multiple equidistant threat clusters, suggesting that adaptive k selection or hierarchical attention mechanisms may be required for extreme-scale scenarios.

3.4.4. Robustness Analysis

The noise simulation experiments introduce Gaussian noise to state observations. A noise level with $\sigma = 5$ m and threat detection range uncertainty of ±20% results in a success rate of 90.8%, attesting to only 3.5% degradation. The redundant computation property built into the multi-head attention mechanism enhances noise robustness; mistakes made within threat evaluation are dampened by weighted combinations. The baselines deteriorate further; DQN drops to 68.4% due to the monolithic state handling.

The error tests in action execution assess the model's control uncertainties by introducing ±5° angular error and ±10% velocity noise within the model's action execution. The suggested solution performs successfully with 91.7% accuracy within these error conditions, proving that there is operational tolerance in the designed solution that safely adapts to moderately uncertain environments. The reason behind its stable accuracy lies within its continuous learning process during training, which includes environment stochasticity to facilitate robust solution learning. Failure analysis shows that most failures occur in areas with higher density and smaller corridors.

4. Discussion

4.1. Analysis of Main Findings

4.1.1. Quantitative Performance Improvement Analysis

The results clearly indicate that there are significant improvements in the performance that can be ascribed to the proposed multi-head attention-based DQN with

dynamic priority replay on various evaluation fronts. The 94.3% mission success rate is an improvement of 17.5% over DQN and 7.1% over the best baseline (Dueling DQN), which also indicates relative improvements of 22.8% and 8.1%, respectively. Notably, this achievement is rather important in the context of challenging penetration missions with complex threat settings. The current state-of-the-art UAV path planning approaches have been surveyed by Ghambari et al. to identify that there are still significant challenges to be met to ensure that either the solution quality is optimal or efficiency in terms of computational requirements is met, which has been successfully addressed with the proposed solution.

The efficacy in improving the path length efficiency over DQN (1247 m vs. 1412 m with 13.2%) exemplifies its stronger trajectory planning capabilities beyond mission accomplishment. A study by Zammit and Van Kampen attempted to identify traditional A* and RRT algorithms that function well in real-time 3D UAV path planning tasks; however, there are computational difficulties in environments with higher density that are alleviated by the application utilizing heuristics that remove redundant searches performed in real-time learning [21]. The threat exposure metric difference with 57.1% over DQN (exposure index 0.12 vs. 0.28) clearly demonstrates better awareness about safety that varies greatly with mission survival rates depending on the proximity to threats in hostile environments.

The degree of convergence acceleration by a factor of 2.2× (612 vs. 1340 episodes) evaluates the learning efficiency benefit attainable with experience prioritization. Better variants of RRT learning techniques demanded significant parameter adjustment depending on the environment [22], while the presented reinforcement learning architecture attains autonomous adjustment without retraining. The computational efficiency analysis substantiates real-time operability with 18.3 ms inference delay to facilitate control rates higher than 50 Hz. The added overhead caused by multi-head attention (18.3 ms vs. 14.7 ms with standard DQN) is tolerable in consideration of the significant learning gain with practical deployability.

4.1.2. Multi-Head Attention Mechanism Analysis

The analysis of the visualization of the learned attention weights shows clear functional specialization among the four parallel attention heads, ensuring the effectiveness of the architecture design. The first attention head always pays most attention to the three closest threats, irrespective of whether they are friendly or hostile, and by how much, incorporating the concept of distance-based prioritization to ensure immediate avoidance in collisions. The weight distributions among this attention head are exponentially decaying with distance, similar to the inverse square law that applies to real collisions. The other attention head shows expertise in lethal attention to distinguish between anti-aircraft fire zones and zones of radar detection, with average weight ratios of 3.2 to 1, even if radars are comparatively closer. Xiang et al. have designed various A* algorithms with more efficacy in path planning tasks with manually configured cost functions [23], and the current attention design is autonomous in grading threats with rewards.

The third attention head exhibits goal-directed behavior, allocating higher weights to threats positioned along the straight-line trajectory toward the objective, effectively identifying obstacles requiring active avoidance versus those naturally avoided by goal approach. Statistical analysis reveals 0.73 Pearson correlation between this head's attention weights and angular deviation from goal direction, confirming trajectory-aware threat assessment. The fourth head captures spatial density patterns through distributed attention across threat clusters rather than individual hazards, with weight entropy 1.8× higher than other heads, indicating less concentrated focus. This density-aware

processing enables recognition of high-threat regions where multiple overlapping detection zones create particularly challenging passages.

Cross-head analysis demonstrates complementary rather than redundant specialization, with average pairwise correlation of only 0.31 between attention weight distributions, confirming that different heads capture orthogonal threat aspects. To quantitatively validate head specialization, we computed the Pearson correlation between each head's attention weights and specific threat features across 1000 test scenarios. Head 1 shows 0.82 correlation with inverse distance (proximity), Head 2 exhibits 0.76 correlation with threat lethality level, Head 3 demonstrates 0.73 correlation with angular deviation from goal direction (alignment), and Head 4 shows 0.68 correlation with local threat count within 200 m radius (density). These correlations significantly exceed the cross-head average of 0.31, providing quantitative evidence for functional specialization rather than redundant representations. Dieumegard et al. addressed UAV trajectory optimization in electromagnetic environments using mathematical programming approaches requiring complete a priori environment knowledge, whereas the multi-head attention learns to process partial observations online [24]. Ablation experiments replacing multi-head with single-head attention of equivalent dimension (256-D output) resulted in 4.9 percentage point success rate drop, validating that parallel specialized processing outperforms monolithic attention. The attention mechanism's interpretability provides operational transparency valuable for mission planning, as analysts can inspect which threats influenced specific maneuver decisions.

4.1.3. Dynamic Priority Impact Analysis

The dynamic priority experience replay process improves the efficiency of sampling experiences with biased learning toward more valuable transitions, which translates to faster convergence and better asymptotic behavior. The results shown by analysis of sampling statistics indicate that priority experiences (top 20% experiences with higher TD error values) account for 58.3% of the experiences among the samples drawn. Gruffeille et al. employed action shaping PPO for radar observability minimization but encountered sample inefficiency in sparse-reward scenarios, a limitation addressed by the priority mechanism's focused learning [25]. The learning acceleration stems from concentrated gradient updates on decision boundaries where small policy improvements yield large value function corrections.

Training stability improvements manifest through reduced reward variance during late-stage learning, with coefficient of variation decreasing from 0.187 for uniform replay to 0.093 for prioritized replay over the final 500 episodes. Lee et al. developed modified A* algorithms for stealth UAV path planning using minimal radar cross-section tactics, achieving deterministic solutions but lacking adaptability to online replanning scenarios where the proposed approach excels through learned policies [26]. The dynamic priority adjustment mechanism preventing indefinite dominance of specific transitions proves essential for maintaining generalization, as priority decay applied to outliers (priorities exceeding $\mu + 3\sigma$) reduces overfitting to peculiar scenarios. Experiments disabling decay resulted in 3.1 percentage point success drop and increased variance, validating the mechanism's contribution to training robustness.

Comparative analysis with alternative prioritization schemes confirms TD-error-based priorities' superiority for value function learning tasks. Mahboubi and Moarref applied particle swarm optimization for UAV path planning under dynamic threats, requiring expensive population evaluations at each replanning cycle, whereas the learned policy provides constant-time inference [27]. The importance sampling correction proves critical for convergence guarantees, as experiments with uncorrected priorities ($\beta = 0$ throughout training) converged to suboptimal policies 4.7 percentage points below the properly

corrected variant. The bias correction exponent annealing schedule from β = 0.4 to β = 1.0 balances fast initial learning against asymptotic optimality, with ablations using fixed β values exhibiting either slower convergence (β = 1.0 from start) or suboptimal final performance (β = 0.4 maintained).

4.2. Method Advantages

4.2.1. Advantages over Traditional Methods

The proposed method shows great benefit over traditional path planning techniques due to its learning capabilities. Classical approaches such as A*, Dijkstra are guaranteed to be optimal if complete environment information is accessible; however, these are computationally intractable tasks with exponentially increasing state space dimensionality in two-dimensional continuous environments. The presented learning-based solution overcomes the curse of dimensionality with the use of artificial neural networks to approximate the value function with constant computational overhead, irrespective of the environment complexity. RRT and RRT* sampling-based techniques ensure probabilistic completeness with suboptimal solutions that need refinement with repeated iterations to improve the solution; on the other hand, solutions are gained from learning with near-optimal results through cumulative experiences. Classical techniques require full environment information prior to task execution and are poorly responsive to partial observability compared to the proposed solution that employs an attention mechanism to process environment observations to make prudent decisions notwithstanding uncertainty.

The key difference lies in the flexibility to cope with dynamic and uncertain environments that render the computational overhead of the replanning process prohibitive in the classical approaches. The classical approaches need to rebuild search trees or compute optimal routes whenever there are changes to the environment, causing delays that are incompatible with real-time requirements in UAV control. The learned policy can generalize well to different threat scenarios with experience-driven feature learning that maintains similar inference speeds, irrespective of environmental changes. The application of deep reinforcement learning was seen to be superior to classical approaches in [28] for the task of swarm UAV wildfire reconnaissance in dynamic environments. The multi-head attention module further adds to the benefits by facilitating emphasis on key features in the environment through learned heuristics that need to be manually defined in classical approaches.

4.2.2. Advantages over Other Deep Reinforcement Learning Methods

Comparative analysis with other deep reinforcement learning models shows that there are key benefits to this proposed value function-based learning solution with architecture modifications. Policy gradient approaches such as PPO are notoriously inefficient in terms of samples in â€œsparse reward penetration tasks, where the successful action sequences must be discovered with the aid of random explorations. The use of the value function-based DQN with experience replay and target networks offers greater efficiency in terms of samples with stable learning targets without degradation over time. The key to converging to optimal solutions lies with approaches that favor replay priority techniques to mitigate these concerns [29]. The application of multi-head attention offers significant improvements over either convolutional networks or recurrent architectures for spatial threat data analysis to yield interpretable feature mappings rather than black-box models.

The dynamic priority replay mechanism distinguishes the proposed approach from both uniform replay DQN and alternative prioritization schemes. While some recent works employ fixed-priority strategies, the proposed dynamic adjustment prevents pathological cases where specific transitions monopolize sampling probability

indefinitely, maintaining diversity essential for generalization. Zou et al. analyzed deep reinforcement learning approaches to anti-jamming communication in UAVs, focusing on the issue of training process stability in hostile environments that are alleviated by dynamic priority functions balancing exploitation and exploration [30]. The decomposed form of state values and action advantage functions in the dueling architecture demonstrates its robustness within penetration planning tasks with many similar actions in safe areas to efficiently spread values. The noisy N-step dueling double deep Q-network with prioritized replay learning was suggested by Zhang et al. to improve navigation tasks in mobile robots and demonstrated the relevance within navigation tasks that this study further extends to multi-head attention models [31].

4.2.3. Practical Application Value

The proposed solution has strong practical applications in performing UAV penetration missions due to various application-oriented properties. The 18.3 ms inference delay is within the real-time requirements to ensure the operationality of the tactical UAV within the usual frequencies of 20–50 Hz without depending on the processing of the ground station and communication that can be intercepted. The robustness properties in terms of handling sensor noise (success rate—90.8% with position uncertainty $\sigma = 5$ m) and action error (success rate—91.7% with action error ±5°) remove the need to consider perfectness in perception/action in the real-world environment. The attention strategy provides useful explanations about the reasoning process behind every action taken during the mission with the aid of mission planning.

Generalization abilities allow policies to be transferred between different operational environments with minimal retraining overhead. The success on new map settings (91.2%) verifies the use of transfer learning to leverage simulation parameters to implement the solution without modifications in the operational environment. The solution's flexibility to accommodate different threat density settings (87.6–98.7% success rates with 5–30 threats) ensures that the solution can be applied to various mission environments that are either permissive or heavily contested. The economic potential of this solution derives from faster convergence (18–24 h) to an optimal state that was previously attainable in weeks with previous architectures. The modular design within the solution ensures that there are no requirements to change architectures in upgrades, such as attention models and prioritization techniques.

4.2.4. Scalability Analysis

The proposed architecture has good scalability properties in more than one dimension that is pertinent to the expanded operational environment. Scalability in computation is achieved through the attention mechanism that processes k-nearest threats, irrespective of the size of the environment. Computational overhead does not increase with expansion of operational ranges. In inference operations, the architecture has stable storage requirements of about 45 MB to compute the trained policy network, making it viable on resource-constrained platforms such as those in tactical UAS applications. Scalability during training is achieved through the use of the prioritized replay method that concentrates on learning during informative experiences. Here, buffers of 100,000 transitions are adequate to permit learning that needs millions of transitions with uniform sampling methods.

The solution can be extended to multi-agent settings using shared policy architectures that involve more than one UAV performing joint penetration maneuvers with attention layers that consider threats from the environment as well as other teammates' locations. In terms of modularity, incorporating other sensors such as radar and fire zone detection can be made easy with the current architecture, with attention heads that can be designed to focus on various types, such as infrared detection, acoustic sensors, and

communication intercept threats. Scalability is possible from four to eight or more attention heads depending on the complexity required by the types of threats; however, practical experimentation has shown that there are diminishing returns beyond the use of six attention heads for today's threats. The representations are transferrable to other types of UAV platforms with differing kinematic constraints to be fine-tuned within 10 to 20% of the original set of training experiences utilizing current policies.

4.3. Limitations and Challenges

4.3.1. Computational Complexity Analysis

The proposed approach faces challenges in terms of computing resources during the training process and prediction tasks. The training time of 18–24 h using high-performance computing resources with RTX 3090 GPUs is significant and economically costly and intensive due to the need to adjust policies to reflect new threats dynamically. The reason behind the higher training time is the significant environment interaction process that needs to be carried out to gain varied experiences with different threats; in other words, each training process simulates about 2.5 million environment steps until completion. The use of the multi-head attention module adds to the computing overhead during training with the simulation of four different attention mappings in parallel to standard DQN models; thus, each step simulates about 28% more time due to the overhead. The significant overhead caused by the proposed architecture during training seems justified by the improved results.

Inference latency of 18.3 ms per action, while fulfilling real-time requirements for most UAV platforms, tends to be close to the limit imposed by the need to respond within less than 20 ms in high-speed tactical missions. The attention module increments the inference time by 3.6 ms due to query-key-value operations and softmax functions on threat representations. The memory access pattern in calculating attention weights shows cache inefficiency due to irregular k-nearest threat feature lookups in k-nearest search operations, potentially introducing bottlenecks on embedded CPUs with shallow cache hierarchies. Methods such as network weight quantization might alleviate the delay issue with lower precision arithmetic; however, initial results indicate that 8-bit weight quantization harms accuracy with 2.3 percentage-point drops in success rates, hinting that precision requirements are significant in attention weight computation. Other means to accelerate inference, such as model trimming and distillation aimed at lower-cost smaller networks, might be useful within strictly resource-limited settings but risk undermining the design foundations that lead to improved performance.

4.3.2. Model Interpretability Issues

Although the attention mechanism ensures that there is explainability over fully black-box models, there still exists opacity with respect to the value function learning process. Although the attention values are informative about the threats that impact the decisions made by the actor-critic agent, there is opacity with respect to the action decisions made by the fully connected layers in the value function network. This makes it difficult to understand how the policy would behave with unforeseen threat combinations that are outside the training data distribution and lead to certification difficulties in critical military scenarios where essential requirements include understanding behavior in all possible scenarios. The features within the layers of the neural networks are without clear semantic meaning and lead to challenges with understanding suboptimal and unsafe behavior during failure analysis within the policy that generates trajectories.

The error in the approximations made by the value function and the effect thereof on the pursuit of real flight trajectories are still difficult to capture rigorously. The nondeterministic environment due to the learning process during residual explorations is

incompatible with the need to maintain deterministic mission planning. Additionally, due to the black-box effect caused by the impact of prioritized replay on convergence processes, there are challenges in tuning hyperparameters since there are no rigorous principles to consider during the choice of prioritized replay parameters, such as the values of exponents. Improved techniques for better understanding and analysis of mission planning logic would be more beneficial.

4.3.3. Practical Deployment Considerations

In practical applications, there are hardware and operational constraints that go beyond those found in laboratory tests. The trained policy neural network has a 45 MB memory footprint and another 128 MB to store activations in inference, which is close to the storage capacity constraints on traditional UAV autopilot platforms with 256 MB or less physical RAM. GPU computing is critical to ensure that the processes remain in real time; however, the reported 18.3 ms inference latency was measured on high-end workstation hardware (RTX 3090). Performance on embedded or aviation-grade processors typical of UAV platforms was not directly evaluated. Preliminary estimates suggest inference latency of 80–120 ms on embedded GPUs, such as NVIDIA Jetson Xavier, which may not satisfy stringent real-time requirements. Additional optimization techniques, including model quantization, pruning, or dedicated neural processing units, would be necessary to achieve comparable real-time performance on flight-certified embedded platforms. The current 2D simulation environment, while capturing essential threat avoidance characteristics, does not fully represent 3D flight dynamics, including altitude variations, climb/descent constraints, and vertical threat coverage. Additionally, simulation-to-reality mismatches in sensor properties and aerodynamic effects may affect real-world performance. However, the proposed multi-head attention mechanism does not inherently depend on the two-dimensional nature of the task. The attention heads process relative threat features such as distance and threat level, which can be extended to 3D by incorporating altitude-related features while maintaining the same computational architecture.

The communication needs involved in the process of downloading updated policies to already operational platforms result in operational risk within contested electromagnetic environments with potential data link jamming and interception threats. The sensitivity of the policy to the hyperparameters' values and initializations requires more than one training process to guarantee convergence with qualification costs added to each updated policy. In real-time applications, violation of the 50 Hz control cycle requirement becomes challenging in worst-case scenarios that include compounded computational delays, potentially violating the 50 Hz cycle requirement, even if average computational delay satisfies the requirement. A GPU inference operational power that consumes 8.3 W within embedded platforms consumes UAV batteries approximately 12% faster than traditional autopilot operational platforms that consume less than 2 W.

4.3.4. Other Outstanding Problems

Some theoretical and application-related issues are still outstanding despite the success achieved by the proposed method. The absence of theoretical convergence proofs for the joint multi-head attention and dynamic priority network architecture obstructs theoretical performance analysis and instead must be empirically tested on various scenario distributions. The robustness of the policy to adversarial situations wherein intelligent adversaries dynamically respond to UAV maneuvers has not been investigated; instead, static placements of threats are assumed during training. While the dynamic threat testing in Section 3.4.2 demonstrates adaptability to moving threats, these tests employ limited and predictable linear movement patterns. Performance against non-linear, deceptive, or reactive threat trajectories that actively anticipate UAV movements remains unvalidated

and may require adversarial training approaches. Additionally, the sensitivity of the proposed method to different threat models remains to be fully investigated. While the current threat parameters are based on typical air defense characteristics, the performance under significantly different threat configurations, such as varying detection probability functions or non-circular coverage patterns, requires further validation. Additionally, challenges surrounding coordination during joint UAV penetration operations within shared airspace need to be investigated; the current ablation study compares multi-head attention against fully connected layers, but does not isolate its effectiveness relative to other state abstraction mechanisms such as graph neural networks or full transformer architectures. Future work should include comparative experiments with graph attention networks for modeling threat relationships and other transformer variants to comprehensively evaluate the architectural choices. Instead, the current one-agent problem formulation overlooks UAV-avoided collisions among friendly entities as well as communication constraints. The usability of the proposed solution to higher-dimensional action spaces to enable fine-grained continuous action control has yet to be determined; instead, the current discrete action problem formulation might impede optimal UAV trajectory smoothness. Long-term autonomous operation-related questions pertain to the potential degradation of the policy during prolonged autonomous operations wherein environment statistics deviate from those during training; instead, approaches to adaptive learning might be warranted. Ethical and legal considerations to enable autonomous lethal weapon use within learned policies must be solved before application to real-world combat operations.

5. Conclusions

5.1. Summary and Main Contributions

This paper presented a novel multi-head attention-enhanced DQN framework with dynamic priority experience replay for UAV penetration path planning in hostile environments. The approach formulates the problem as an MDP and employs four specialized attention heads to process threat information in parallel, enabling selective focus on proximity, lethality, trajectory alignment, and spatial density features. The dynamic priority mechanism accelerates learning by adaptively adjusting sampling strategies throughout training.

Experimental results demonstrate significant performance improvements: 94.3% mission success rate (7.1–17.5% higher than baselines), 2.2× faster convergence, and 57.1% reduction in threat exposure. Ablation studies confirm the synergistic effects of multi-head attention (6.7% contribution) and dynamic priority (5.4% contribution). The method exhibits strong robustness in high-threat environments, maintaining 87.6% success rate with 20–30 threats while meeting real-time requirements (18.3 ms inference latency). These results validate the practical viability of the proposed approach for autonomous UAV penetration missions.

5.2. Future Work

Future research directions include extending the framework to multi-agent coordination for collaborative penetration missions with shared policy networks, extending the framework to 3D environments with realistic flight dynamics and altitude constraints, implementing hardware-in-the-loop testing and real-world validation to address simulation-to-reality gaps, developing theoretical convergence guarantees for the joint attention-priority architecture, and exploring meta-learning techniques for rapid adaptation to novel threat types with minimal fine-tuning. The methodology could be adapted to other

autonomous navigation tasks including search-and-rescue, infrastructure inspection, and urban delivery operations.

Author Contributions: L.C. completed the work and contributed to the writing of the manuscript. W.S. conceived the project, provided the research methodology, and ensured the quality of the methodology. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China (No. 61976083) and Hubei Province Key R&D Program of China (No. 2022BBA0016).

Data Availability Statement: All data generated or analyzed during this study are included in this published article.

Acknowledgments: The authors would like to thank Wei Shang for his early proposal for this manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Luo, J.; Tian, Y.; Wang, Z. Research on unmanned aerial vehicle path planning. *Drones* **2024**, *8*, 51. https://doi.org/10.3390/drones8020051.
2. Wang, F.; Zhu, X.; Zhou, Z.; Tang, Y. Deep-reinforcement-learning-based UAV autonomous navigation and collision avoidance in unknown environments. *Chin. J. Aeronaut.* **2024**, *37*, 237–257. https://doi.org/10.1016/j.cja.2023.09.033.
3. Debnath, D.; Vanegas, F.; Boiteau, S.; Gonzalez, F. An integrated geometric obstacle avoidance and genetic algorithm tsp model for uav path planning. *Drones* **2024**, *8*, 302. https://doi.org/10.3390/drones8070302.
4. Ghambari, S.; Golabi, M.; Jourdan, L.; Lepagnot, J.; Idoumghar, L. UAV path planning techniques: A survey. *RAIRO-Oper. Res.* **2024**, *58*, 2951–2989.
5. Wang, Z.; Song, S.; Cheng, S. Path planning of mobile robot based on improved double deep Q-network algorithm. *Front. Neurorobotics* **2025**, *19*, 1512953. https://doi.org/10.3389/fnbot.2025.1512953.
6. Dong, R.; Pan, X.; Wang, T.; Chen, G. UAV path planning based on deep reinforcement learning. In *Artificial Intelligence for Robotics and Autonomous Systems Applications*; Springer: Cham, Switzerland, 2023; pp. 27–65. https://doi.org/10.1007/978-3-031-28715-2_2.
7. Guo, Z.; Chen, H.; Li, S. Deep Reinforcement Learning-Based UAV Path Planning for Energy-Efficient Multitier Cooperative Computing in Wireless Sensor Networks. *J. Sens.* **2023**, *2023*, 2804943. https://doi.org/10.1155/2023/2804943.
8. Tang, J.; Liang, Y.; Li, K. Dynamic scene path planning of UAVs based on deep reinforcement learning. *Drones* **2024**, *8*, 60. https://doi.org/10.3390/drones8020060.
9. Xu, Y.; Wei, Y.; Jiang, K.; Wang, D.; Deng, H. Multiple UAVs path planning based on deep reinforcement learning in communication denial environment. *Mathematics* **2023**, *11*, 405. https://doi.org/10.3390/math11020405.
10. Liu, X.; Liu, H.; Zheng, K.; Liu, J.; Taleb, T.; Shiratori, N. AoI-Minimal Clustering, Transmission and Trajectory Co-Design for UAV-Assisted WPCNs. *IEEE Trans. Veh. Technol.* **2024**, *74*, 1035–1051.
11. Hu, K.; Xu, K.; Xia, Q.; Li, M.; Song, Z.; Song, L.; Sun, N. An overview: Attention mechanisms in multi-agent reinforcement learning. *Neurocomputing* **2024**, *598*, 128015. https://doi.org/10.1016/j.neucom.2024.128015.
12. Wang, G.; Ding, J.; Hu, F. Deep Reinforcement Learning Recommendation System Algorithm Based on Multi-Level Attention Mechanisms. *Electronics* **2024**, *13*, 4625. https://doi.org/10.3390/electronics13234625.
13. Qian, F.; Su, K.; Liang, X.; Zhang, K. Task assignment for UAV swarm saturation attack: A deep reinforcement learning approach. *Electronics* **2023**, *12*, 1292. https://doi.org/10.3390/electronics12061292.
14. Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized experience replay. *arXiv* **2015**, arXiv:1511.05952. https://doi.org/10.48550/arXiv.1511.05952.
15. Yan, W.-z.; Li, X.-h.; Ding, Y.-m.; He, J.; Cai, B. DQN with prioritized experience replay algorithm for reducing network blocking rate in elastic optical networks. *Opt. Fiber Technol.* **2024**, *82*, 103625. https://doi.org/10.1016/j.yofte.2023.103625.
16. Yin, Y.; Wang, Z.; Zheng, L.; Su, Q.; Guo, Y. Autonomous UAV navigation with adaptive control based on deep reinforcement learning. *Electronics* **2024**, *13*, 2432. https://doi.org/10.3390/electronics13132432.

17. Chao, Y.; Dillmann, R.; Roennau, A.; Xiong, Z. E-DQN-based path planning method for drones in Airsim simulator under unknown environment. *Biomimetics* **2024**, *9*, 238. https://doi.org/10.3390/biomimetics9040238.
18. Zhang, D.; Xuan, Z.; Zhang, Y.; Yao, J.; Li, X.; Li, X. Path planning of unmanned aerial vehicle in complex environments based on state-detection twin delayed deep deterministic policy gradient. *Machines* **2023**, *11*, 108. https://doi.org/10.3390/machines11010108.
19. Gebreyesus, G.D.; Fellek, G.T.; Farid, A.; Qiang, Y.; Fujimura, S.; Yoshie, O. A Deep Reinforcement Learning Framework with Convolution Augmented Attention and Gate Mechanism for Job Shop Scheduling. In Proceedings of the 2024 International Conference on Innovation in Artificial Intelligence, Tokyo, Japan, 16–18 March 2024; pp. 171–177. https://doi.org/10.1145/3655497.3655512.
20. Gök, M. Dynamic path planning via Dueling Double Deep Q-Network (D3QN) with prioritized experience replay. *Appl. Soft Comput.* **2024**, *158*, 111503. https://doi.org/10.1016/j.asoc.2024.111503.
21. Zammit, C.; Van Kampen, E.-J. Comparison of a* and rrt in real–time 3d path planning of uavs. In Proceedings of the Aiaa scitech 2020 forum, Orlando, FL, USA, 6–10 January 2020; p. 0861. https://doi.org/10.2514/6.2020-0861.
22. Zhang, R.; Guo, H.; Andriukaitis, D.; Li, Y.; Królczyk, G.; Li, Z. Intelligent path planning by an improved RRT algorithm with dual grid map. *Alex. Eng. J.* **2024**, *88*, 91–104. https://doi.org/10.1016/j.aej.2023.12.044.
23. Sun, Y.; Yuan, Q.; Gao, Q.; Xu, L. A Multiple Environment Available Path Planning Based on an Improved A* Algorithm. *Int. J. Comput. Intell. Syst.* **2024**, *17*, 172. https://doi.org/10.1007/s44196-024-00571-z.
24. Atayev, A.; Fliege, J.; Zemkoho, A. Trajectory optimization of unmanned aerial vehicles in the electromagnetic environment. *Optim. Eng.* **2025**, *26*, 159–198. https://doi.org/10.1007/s11081-024-09893-5.
25. Ali, A.M.; Perrusquía, A.; Guo, W.; Tsourdos, A. Flight plan optimisation of unmanned aerial vehicles with minimised radar observability using action shaping proximal policy optimisation. *Drones* **2024**, *8*, 546.
26. Zhang, Z.; Jiang, J.; Wu, J.; Zhu, X. Efficient and optimal penetration path planning for stealth unmanned aerial vehicle using minimal radar cross-section tactics and modified A-Star algorithm. *ISA Trans.* **2023**, *134*, 42–57. https://doi.org/10.1016/j.isatra.2022.07.032.
27. Shin, J.-J.; Bang, H. UAV path planning under dynamic threats using an improved PSO algorithm. *Int. J. Aerosp. Eng.* **2020**, *2020*, 8820284. https://doi.org/10.1155/2020/8820284.
28. Demir, K.; Tumen, V.; Kosunalp, S.; Iliev, T. A deep reinforcement learning algorithm for trajectory planning of swarm uav fulfilling wildfire reconnaissance. *Electronics* **2024**, *13*, 2568. https://doi.org/10.3390/electronics13132568.
29. Huang, H.; Li, Y.; Song, G.; Gai, W. Deep reinforcement learning-driven UAV data collection path planning: A study on minimizing AoI. *Electronics* **2024**, *13*, 1871. https://doi.org/10.3390/electronics13101871.
30. Zou, C.; Li, C.; Li, Y.; Yan, X. RIS-assisted robust beamforming for UAV anti-jamming and eavesdropping communications: A deep reinforcement learning approach. *Electronics* **2023**, *12*, 4490. https://doi.org/10.3390/electronics12214490.
31. Hu, W.; Zhou, Y.; Ho, H.W. Mobile Robot Navigation Based on Noisy N-Step Dueling Double Deep Q-Network and Prioritized Experience Replay. *Electronics* **2024**, *13*, 2423. https://doi.org/10.3390/electronics13122423.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.